ASP.NET MVC Interview Questions & Answers

> *Click :star:if you like the project. Pull Request are highly appreciated. Follow me @kansiris87 for technical updates.*

Table of Contents

No.	Questions
1	What is MVC?
2	Explain MVC design pattern?
3	What is Domain Driven Design and Development?
4	What is MVP pattern?
5	What is MVVM pattern?
6	What is ASP.NET MVC?
7	How MVC pattern works in ASP.NET MVC?
8	How Model, View and Controller communicate with each other in ASP.NET MVC?
9	What are advantages of ASP.NET MVC?
10	Explain brief history of ASP.NET MVC?
11	What is difference between 3-layer architecture and MVC architecture?
12	What is difference between ASP.NET WebForm and ASP.NET MVC?
13	What is ViewModel in ASP.NET MVC?
14	Explain ASP.NET MVC pipeline?
15	What is Routing in ASP.NET MVC?
16	How to define a route in ASP.NET MVC?
17	What is Attribute Routing and how to define it?
18	When to use Attribute Routing?
19	How to enable Attribute Routing in ASP.NET MVC?
20	How to define Attribute Routing for Area in ASP.NET MVC?
21	What is difference between Routing and URL Rewriting?
22	What is Route Constraints in ASP.NET MVC?
23	How route table is created in ASP.NET MVC?
24	What are important namespaces in ASP.NET MVC?
25	What is View Engine?
26	How View Engine works?

27	What is Razor View Engine?
28	How to make Custom View Engine?
29	How to register Custom View Engine in ASP.NET MVC?
30	Can you remove default View Engine in ASP.NET MVC?
31	What is difference between Razor and WebForm engine?
32	What are HTML Helpers in ASP.NET MVC?
33	What are different types of HTML Helpers?
34	What are Url Helpers?
35	What is Validation Summary?
36	What are AJAX Helpers?
37	What is unobtrusive AJAX?
38	What are various configuration options for AJAX Helpers?
39	What is Cross Domain AJAX?
40	What are Layouts in ASP.NET MVC?
41	What are Sections in ASP.NET MVC?
42	What are RenderBody and RenderPage in ASP.NET MVC?
43	What are Styles.Render and Scripts.Render?
44	How to enable and disable optimizations in ASP.NET MVC?
45	What is ViewStart?
46	When to use _ViewStart?
47	What are different ways of rendering layout in ASP.NET MVC?
48	What is App_Start folder in ASP.NET MVC?
49	What are different ways of returning/rendering a view in ASP.NET MVC?
50	What are differences among ViewData, ViewBag, TempData and Session?
51	How to persist data in TempData?
52	How to control Session behavior in ASP.NET MVC?
53	How TempData is related to Session in ASP.NET MVC?
54	What are Action methods in ASP.NET MVC?
55	What is ActionResult and how is it different from others?
56	How to make a Non-Action method in ASP.NET MVC?

57	Can you change action method name?
58	How to restrict an action method to be invoked only by HTTP GET, POST, PUT or DELETE?
59	How to determine an action method is invoked by HTTP GET or POST?
60	How to determine an AJAX request?
61	What is Data Annotations in ASP.NET MVC?
62	How to apply Server side validation in ASP.NET MVC?
63	How to determine there is no error in Model State?
64	How to enable and disable client-side validation in ASP.NET MVC?
65	What is a CDN and advantages of CDN?
66	What is jquery.validate.unobtrusive.js?
67	What is Bundling and Minification in ASP.NET MVC?
68	Can we use Bundling and Minification in ASP.NET MVC3 or ASP.NET4.0?
69	How Bundling use browser Cache capability?
70	What is Partial View in ASP.NET MVC?
71	How do you return a partial view from controller?
72	What are different ways of rendering a Partial View in ASP.NET MVC?
73	What is Area in ASP.NET MVC?
74	How to register Area in ASP.NET MVC?
75	What is Child action and how to invoke it?
76	What is Scaffolding?
77	How Scaffold templates works in ASP.NET MVC?
78	What are ASP.NET MVC Filters and Attributes?
79	What are different types of Filters in ASP.NET MVC?
80	When Exception filters are executed in ASP.NET MVC?
81	What is the order of execution of filters in ASP.NET MVC?
82	How to configure filters in ASP.NET MVC?
83	How Authentication and Authorization work in ASP.NET MVC?
84	How Forms Authentication and Authorization work in ASP.NET MVC?
85	How to implement custom Forms Authentication and Authorization in MVC?
86	How to allow HTML tags in ASP.NET MVC?

87	What is caching and when to use it?
88	What are advantages of caching?
89	What is output caching?
90	What is Donut caching and Donut hole caching in ASP.NET MVC?
91	What is loose coupling and how is it possible?
92	What are Dependency Inversion Principle (DIP) and IoC?
93	What is Dependency Injection (DI)?
94	What is Service Locator?
95	What are different ways to implement Dependency Injection (DI)?
96	What are advantages of Dependency Injection (DI)?
97	What is IoC or DI container?
98	What are popular DI containers?
99	What is Test Driven Development (TDD)?
100	What are commonly used tool for Unit Testing in ASP.NET MVC?

ASP.NET MVC

What is MVC (Model view controller)?

Model–view–controller (MVC) is a software architectural pattern for implementing user interfaces. It divides a given software application into three interconnected parts, so as to separate internal representation of information from the way that information is presented to or accepted from the user.

The ASP.NET MVC framework provides an alternative to the ASP.NET Web Forms pattern for creating web applications. The ASP.NET MVC Framework is a lightweight, highly testable presentation framework that (as with Web Forms-based applications) is integrated with existing ASP.NET features, such as master pages and membership-based authentications. The MVC framework is defined in the System.Web.Mvc assembly. It provides full control over HTML, JavaScript and CSS. It's the better as well as a recommended approach for large-scale applications where various teams are working together.

MVC is a framework for building web applications using a MVC (Model View Controller) design:

• The Model represents the application core (for instance a list of database records).

• The View displays the data (the database records).

• The Controller handles the input (to the database records).

The MVC model also provides full control over HTML, CSS, and JavaScript.

The MVC model defines web applications with 3 logic layers:

• The business layer (Model logic)

• The display layer (View logic)

• The input control (Controller logic)

The Model is the part of the application that handles the logic for the application data. Often model objects retrieve data (and store data) from a database.

The View is the part of the application that handles the display of the data. Most often the views are created from the model data. The Controller is the part of the application that handles user interaction. Typically controllers read data from a view, control user input, and send input data to the model.

The MVC separation helps you manage complex applications, because you can focus on one aspect a time. For example, you can focus on the view without depending on the business logic. It also makes it easier to test an application.

What are the advantages of MVC?

Multiple view support - Due to the separation of the model from the view, the user interface can display multiple views of the same data at the same time.

Change Accommodation - User interfaces tend to change more frequently than business rules (different colors, fonts, screen layouts, and levels of support for new devices such as cell phones or PDAs) because the model does not depend on the views, adding new a type of views to the system generally does not affect the model. As a result, the scope of change is confined to the view.

SoC – Separation of Concerns - Separation of Concerns is one of the core advantages of ASP.NET MVC. The MVC framework provides a clean separation of the UI, Business Logic, Model or Data.

More Control - The ASP.NET MVC framework provides more control over HTML, JavaScript and CSS than the traditional Web Forms.

Testability - ASP.NET MVC framework provides better testability of the Web Application and good support for the test driven development too.

Lightweight - ASP.NET MVC framework doesn't use View State and thus reduces the bandwidth of the requests to an extent. Full features of ASP.NET - One of the key advantages of using ASP.NET MVC is that it is built on top of ASP.NET framework and hence most of the features of the ASP.NET like membership providers, roles, etc can still be used.

Explain MVC application life cycle?

Any web application has two main execution steps, first understanding the request and depending on the type of the request sending out appropriate response. MVC application life cycle is not different it has two main phases, first creating the request object and second sending our response to the browser.

Creating the request object: The request object creation has four major steps. The following is the detailed explanation of the same(short cut to remember FFRCAR)

Step 1: Fill route

MVC requests are mapped to route tables which in turn specify which controller and action to be invoked. So if the request is the first request the first thing is to fill the route table with routes collection. This filling of route table happens in the global.asax file.

Step 2: Fetch route

Depending on the URL sent "UrlRoutingModule" searches the route table to create "RouteData" object which has the details of which controller and action to invoke.

Step 3: Request context created

The “RouteData” object is used to create the “RequestContext” object.

Step 4: Controller instance created

This request object is sent to “MvcHandler” instance to create the controller class instance. Once the controller class object is created it calls the “Execute” method of the controller class.

Creating Response object: This phase has two steps executing the action and finally sending the response as a result to the view.

List out different return types of a controller action method?

Controller actions are methods defined in the controller class and responsible to perform required operations on the user's inputs like as form values, query strings values etc. with the help of Model and passing the results back to the View. There are total nine return types we can use to return results from controller to view. The base type of all these result types is ActionResult.

ViewResult (View): This return type is used to return a webpage from an action method.Returns a ViewResult which renders the specified or default view by using controller View() helper method

PartialviewResult (Partialview): This return type is used to send a part of a view which will be rendered in another view. or Returns a PartialViewResult which renders the specified or default partial view (means a view without its layout) by using controller PartialView() helper method.

RedirectResult (Redirect): This return type is used to redirect to any other controller and action method depending on the URL.or. Returns a RedirectResult which Issues an HTTP 301 or 302 redirection to a specific URL by using controller Redirect() helper method. RedirectToRouteResult (RedirectToAction, RedirectToRoute): This return type is used when we want to redirect to any other action method. or. Returns a RedirectToRouteResult which Issues an HTTP 301 or 302 redirection to an action method or specific route entry by using controller RedirectToAction(), RedirectToActionPermanent(), RedirectToRoute(), RedirectToRoutePermanent() helper methods.

ContentResult (Content): This return type is used to return HTTP content type like text/plain as the result of the action. or . Returns a ContentResult which renders raw text like as "Hello, DotNet Tricks!" by using controller Content() helper method.

jsonResult (json): This return type is used when we want to return a JSON message. or. Returns a JsonResult which serializes an object in JSON format (like as "{ "Message": Hello, World! }") and renders it by using controller Json() helper method.

javascriptResult (javascript): This return type is used to return JavaScript code that will run in browser. or. Returns a JavaScriptResult which renders a snippet of JavaScript code like as "function hello() { alert(Hello, World!); }" by using controller JavaScript() helper method. This is used only in AJAX scenarios.

FileResult (File): This return type is used to send binary output in response. or.Returns a FileResult which renders the contents of a file like as PDF, DOC, Excel etc. by using controller File() helper method.

EmptyResult: This return type is used to return nothing (void) in the result. or. Returns no result returned by an action. This has no controller helper method.

HttpNotFoundResult - Returns an HttpNotFoundResult which renders a 404 HTTP Status Code response by using controller HttpNotFound() helper method.

HttpUnauthorizedResult - Returns an HttpUnauthorizedResult which renders a 401 HTTP Status Code (means "not authorized") response. This has no controller helper method. This is used for authentication (forms authentication or Windows authentication) to ask the user to log in.

HttpStatusCodeResult - Returns an HttpStatusCodeResult which renders a specified HTTP code response. This has no controller helper method.

What are Filters in MVC?

In MVC, controllers define action methods and these action methods generally have a one-to-one relationship with UI controls such as clicking a button or a link, etc. For example, in one of our previous examples, the UserController class contained methods UserAdd, UserDelete, etc. But many times we would like to perform some action before or after a particular operation. For achieving this functionality, ASP.NET MVC provides feature to add pre and post action behaviors on controller's action methods.

Types of Filters: ASP.NET MVC framework supports the following action filters:

Action Filters: Action filters are used to implement logic that gets executed before and after a controller action executes. We will look at Action Filters in detail in this chapter.

Authorization Filters: Authorization filters are used to implement authentication and authorization for controller actions. Result Filters: Result filters contain logic that is executed before and after a view result is executed. For example, you might want to modify a view result right before the view is rendered to the browser.

Exception Filters: Exception filters are the last type of filter to run. You can use an exception filter to handle errors raised by either your controller actions or controller action results. You can also use exception filters to log errors.

Action filters are one of most commonly used filters to perform additional data processing, or manipulating the return values or cancelling the execution of action or modifying the view structure at run time.

What are Action Filters in MVC?

Action Filters are additional attributes that can be applied to either a controller section or the entire controller to modify the way in which action is executed. These attributes are special .NET classes derived from System.Attribute which can be attached to classes, methods, properties and fields.

ASP.NET MVC provides the following action filters:

Output Cache: This action filter caches the output of a controller action for a specified amount of time.

Handle Error: This action filter handles errors raised when a controller action executes.

Authorize: This action filter enables you to restrict access to a particular user or role.

Now we will see the code example to apply these filters on an example controller ActionFilterDemoController.

(ActionFilterDemoController is just used as an example. You can use these filters on any of your controllers.)

Output Cache: E.g.: Specifies the return value to be cached for 10 seconds.

publicclassActionFilterDemoController: Controller

{ [HttpGet]

OutputCache(Duration = 10)]

publicstringIndex()

{ returnDateTime.Now.ToString("T"); } }

Explain what is routing in MVC? What are the three segments for routing important?

Routing is a mechanism to process the incoming url that is more descriptive and give desired response. In this case, URL is not mapped to specific files or folder as was the case of earlier day's web sites.

There are two types of routing (after the introduction of ASP.NET MVC 5).

Convention based routing: to define this type of routing, we call MapRoute method and set its unique name, URL pattern and specify some default values.

Attribute based routing: to define this type of routing, we specify the Route attribute in the action method of the controller.

Routing is the URL pattern that is mappped together to a handler,rounting is responsible for incoming browser request for particular MVC controller. In other ways let us say routing help you to define a URL structure and map the URL with controller. There are three segments for routing that are important,

1. ControllerName
2. ActionMethodName
3. Parammeter

i.e: ControllerName/ActionMethodName/{ParamerName} and also route map coding written in a Global.asax file.

What is Route in MVC? What is Default Route in MVC?

A route is a URL pattern that is mapped to a handler. The handler can be a physical file, such as an .aspx file in a Web Forms application. A handler can also be a class that processes the request, such as a controller in an MVC application. To define a route, you create an instance of the Route class by specifying the URL pattern, the handler, and optionally a name for the route. You add the route to the application by adding the Route object to the static Routes property of the RouteTable class. The Routesproperty is a RouteCollection object that stores all the routes for the application. You typically do not have to write code to add routes in an MVC application. Visual Studio project templates for MVC include preconfigured URL routes. These are defined in the Mvc Application class, which is defined in the Global.asax file.

Route definition

Example of matching URL

{controller}/{action}/{id}

/Products/show/beverages

{table}/Details.aspx

/Products/Details.aspx

blog/{action}/{entry}

/blog/show/123

{reporttype}/{year}/{month}/{day}

/sales/2008/1/5

{locale}/{action}

/US/show

{language}-{country}/{action}

/en-US/show

Default Route:

The default ASP.NET MVC project templates add a generic route that uses the following URL convention to break the URL for a given request into three named segments.

URL: "{controller}/{action}/{id}"

This route pattern is registered via call to the MapRoute() extension method of RouteCollection.

Mention what is the difference between Temp data, View, Session and View Bag?

Or

How can we pass the data From Controller to View in MVC?

In ASP.NET MVC there are three ways to pass/store data between the controllers and views.

ViewData:

ViewData is used to pass data from controller to view.

It is derived from ViewDataDictionary class.

It is available for the current request only.

Requires typecasting for complex data type and checks for null values to avoid error.

If redirection occurs, then its value becomes null.

ViewBag:

ViewBag is also used to pass data from the controller to the respective view.

ViewBag is a dynamic property that takes advantage of the new dynamic features in C# 4.0

It is also available for the current request only.

If redirection occurs, then its value becomes null.

Doesn't require typecasting for complex data type.

TempData:

TempData is derived from TempDataDictionary class

TempData is used to pass data from the current request to the next request

It keeps the information for the time of an HTTP Request. This means only from one page to another. It helps to maintain the data when we move from one controller to another controller or from one action to another action

It requires typecasting for complex data type and checks for null values to avoid error. Generally, it is used to store only one time messages like the error messages and validation messages.

Session

Session is also used to pass data within the ASP.NET MVC application and Unlike TempData, it never expires.

Session is valid for all requests, not for a single redirect.

It's also required typecasting for getting data and check for null values to avoid error.

What is Partial View in MVC?

A partial view is a chunk of HTML that can be safely inserted into an existing DOM. Most commonly, partial views are used to componentize Razor views and make them easier to build and update. Partial views can also be returned directly from controller methods. In this case, the browser still receives text/html content but not necessarily HTML content that makes up an entire page. As a result, if a URL that returns a partial view is directly invoked from the address bar of a browser, an incomplete page may be displayed. This may be something like a page that misses title, script and style sheets.

However, when the same URL is invoked via script, and the response is used to insert HTML within the existing DOM, then the net effect for the end user may be much better and nicer. Partial view is a reusable view (like a user control) which can be embedded inside other view. For example, let's say all the pages of your site have a standard structure with left menu, header, and footer as in the following image,

Explain what is the difference between View and Partial View?

View:

• It contains the layout page.

• Before any view is rendered, viewstart page is rendered.

• View might have markup tags like body, html, head, title, meta etc.

• View is not lightweight as compare to Partial View.

Partial View:

It does not contain the layout page.

Partial view does not verify for a viewstart.cshtml.We cannot put common code for a partial view within the viewStart.cshtml.page.

Partial view is designed specially to render within the view and just because of that it does not consist any mark up.

We can pass a regular view to the RenderPartial method.

What are HTML Helpers in ASP.NET MVC?

An HTML Helper is just a method that returns a HTML string. The string can represent any type of content that you want. For example, you can use HTML Helpers to render standard HTML tags like HTML

and tags etc.

You can also create your own HTML Helpers to render more complex content such as a menu strip or an HTML table for displaying database data.

What are different types of HTML Helpers?

There are three types of HTML helpers as given below:

Inline Html Helpers - These are create in the same view by using the Razor @helper tag. These helpers can be reused only on the same view.

```
@helper ListingItems(string[] items)
{
@foreach (string item in items)
{
@item
}
}
```

1.

Programming Languages:

@ListingItems(new string[] { "C", "C++", "C#" })

Book List:

@ListingItems(new string[] { "How to C", "how to C++", "how to C#" })

Built-In Html Helpers - Built-In Html Helpers are extension methods on the HtmlHelper class. The Built-In Html helpers can be divided into three categories-

Standard Html Helpers - These helpers are used to render the most common types of HTML elements like as HTML text boxes, checkboxes etc. A list of most common standard html helpers is given below:

HTML Element Example

TextBox @Html.TextBox("Textbox1", "val")

Output: val

TextArea @Html.TextArea("Textarea1", "val", 5, 15, null)

Output:

val

Password @Html.Password("Password1", "val")

output: •••

Hidden Field @Html.Hidden("Hidden1", "val")

Output:

Output:

CheckBox @Html.CheckBox("Checkbox1", false)

Output: ☐

RadioButton @Html.RadioButton("Radiobutton1", "val", true)

Output: <input checked="checked" id="Radiobutton1" name="Radiobutton1"type="radio" value="val" />

Drop-down list @Html.DropDownList ("DropDownList1", new SelectList(new [] {"Male","Female"}))

Output: ⌄

M
F

Multiple-select Html.ListBox("ListBox1", new MultiSelectList(new [] {"Cricket", "Chess"}))

Output: Cricket

Chess

Strongly Typed HTML Helpers - These helpers are used to render the most common types of HTML elements in strongly typed view like as HTML text boxes, checkboxes etc. The HTML elements are created based on model properties.

The strongly typed HTML helpers work on lambda expression. The model object is passed as a value to lambda expression, and you can select the field or property from model object to be used to set the id, name and value attributes of the HTML helper. A list of most common strongly-typed html helpers is given below:

HTML Element Example

TextBox @Html.TextBoxFor(m=>m.Name)

Output: Name-val

TextArea @Html.TextArea(m=>m.Address , 5, 15, new{}))

Output: Addressvalue

Password @Html.PasswordFor(m=>m.Password)

Output:

Hidden Field @Html.HiddenFor(m=>m.UserId)

Output:

CheckBox @Html.CheckBoxFor(m=>m.IsApproved)

Output: ☐

RadioButton @Html.RadioButtonFor(m=>m.IsApproved, "val")

Output:

Drop-down list @Html.DropDownListFor(m => m.Gender, new SelectList(new [] {"Male","Female"}))

Output:

Male
Female

Multiple-select Html.ListBoxFor(m => m.Hobbies, new MultiSelectList(new [] {"Cricket","Chess"}))

Output:

Chess

Templated HTML Helpers - These helpers figure out what HTML elements are required to render based on properties of your model class. This is a very flexible approach for displaying data to the user, although it requires some initial care and attention to set up. To setup proper HTML element with Templated HTML Helper, make use of DataType attribute of DataAnnitation class.

For example, when you use DataType as Password, A templated helper automatically render Password type HTML input element. Templated Helper Example

Display

Renders a read-only view of the specified model property and selects an appropriate HTML element based on property's data type and metadata.Html.Display("Name")

DisplayFor

Strongly typed version of the previous helper Html.DisplayFor(m => m. Name)

Editor Renders an editor for the specified model property and selects an appropriate HTML element based on property's data type and metadata Html.Editor("Name").

EditorFor Strongly typed version of the previous helper Html.EditorFor(m => m. Name)

Custom Html Helpers - You can also create your own custom helper methods by creating an extension method on the HtmlHelper class or by creating static methods with in a utility class.

public static class CustomHelpers { //Submit Button Helper public static MvcHtmlString SubmitButton(this HtmlHelper helper, string buttonText) {string str = "<input type="submit" value="" + buttonText + ""/>"; return new MvcHtmlString(str);} //Readonly Strongly-Typed TextBox Helper public static MvcHtmlString TextBoxFor<TModel, TValue>(this HtmlHelper htmlHelper, Expression<Func<TModel, TValue>>expression, bool isReadonly) {MvcHtmlString html = default(MvcHtmlString); if (isReadonly) {html = System.Web.Mvc.Html.InputExtensions.TextBoxFor(htmlHelper, expression, new { @class = "readOnly",@readonly = "read-only" }); } else {html = System.Web.Mvc.Html.InputExtensions.TextBoxFor(htmlHelper, expression);} return html;}}

What are Url Helpers?

Url helpers allows you to render HTML links and raw URLs. The output of these helpers is dependent on the routing configuration of your ASP.NET MVC application.

HTML Element Example

Relative URL @Url.Content("~/Files/asp.netmvc.pdf")

Output: /Files/asp.netmvc.pdf

Based on @Html.ActionLink("About Us", "About", "Home")

Output: About Us

action/controller @Html.ActionLink("About Me", "About", "Home", "http","www.kansiris.org",null,null,null)

Output: About Me

Raw URL for Action Url.Action("About", "Home")

Output: /Home/About

Explain attribute based routing in MVC?

In ASP.NET MVC 5.0 we have a new attribute route, By using the "Route" attribute we can define the URL structure. For example in the below code we have decorated the "GotoAbout" action with the route attribute. The route attribute says that the "GotoAbout" can be invoked using the URL structure "Users/about". Code: public class HomeController: Controller { [Route("Users/about")] publicActionResultGotoAbout() { return View(); } }

What is TempData in MVC?

TempData is a dictionary object to store data temporarily. It is a TempDataDictionary class type and instance property of the Controller base class. TempData is able to keep data for the duration of a HTP request, in other words it can keep live data between two consecutive HTTP requests. It will help us to pass the state between action methods. TempData only works with the current and subsequent request. TempData uses a session variable to store the data. TempData Requires type casting when used to retrieve data. TempDataDictionary is inherited from the IDictionary<string, object>, ICollection<KeyValuePair<string, object>>,

IEnumerable<KeyValuePair<string, object>> and IEnumerable interfaces.

Example: public ActionResult FirstRequest() { List < string > TempDataTest = new List < string > (); TempDataTest.Add("Tejas"); TempDataTest.Add("Jignesh"); TempDataTest.Add("Rakesh"); TempData["EmpName"] = TempDataTest; return View(); }

```
public ActionResult ConsecutiveRequest()
{       List < string > modelData = TempData["EmpName"] as
                List < string > ;
    TempData.Keep();  return View(modelData);       }
```

What is Razor in MVC? Or What is Razor View Engine?

Razor Engine is an advanced view engine that was introduced with MVC3. This is not a new language but it is a new markup syntax. Razor has new and advance syntax that are compact, expressive and reduces typing. Razor syntax are easy to learn and much clean than Web Form syntax. Razor uses @ symbol to write markup as: @Html.ActionLink("SignUp", "SignUp")

Why is Razor?

Compact & Expressive. Razor minimizes the number of characters and keystroke required in a file, and enables a fast coding workflow. Unlike most template syntaxes, you do not need to

file, and enables a fast coding workflow. Unlike most template syntaxes, you do not need to interrupt your coding to explicitly denote server blocks within your HTML. The parser is smart enough to infer this from your code. This enables a really compact and expressive syntax which is clean, fast and fun to type.

Easy to Learn: Razor is easy to learn and enables you to quickly be productive with a minimum of effort. We can use all your existing language and HTML skills.

Works with any Text Editor: Razor doesn't require a specific tool and enables you to be productive in any plain old text editor (notepad works great).

Has great Intellisense.

Unit Testable: The new view engine implementation will support the ability to unit test views (without requiring a controller or web-server, and can be hosted in any unit test project - no special app-domain required).

Differences between Razor and ASPX View Engine in MVC?

Razor View Engine VS ASPX View Engine:

Razor View Engine ASPX View Engine (Web form view engine) The namespace used by the Razor View Engine is System.Web.Razor The namespace used by the ASPX View Engine is System.Web.Mvc.WebFormViewEngine The file extensions used by the Razor View Engine are different from a web form view engine. It uses cshtml with C# and vbhtml with vb for views, partial view, templates and layout pages. The file extensions used by the Web Form View Engines are like ASP.Net web forms. It uses the ASPX extension to view the aspc extension for partial views or User Controls or templates and master extensions for layout/master pages. The Razor View Engine is an advanced view engine that was introduced with MVC 3.0. This is not a new language but it is markup. A web form view engine is the default view engine and available from the beginning of MVC Razor has a syntax that is very compact and helps us to reduce typing. The web form view engine has syntax that is the same as an ASP.Net forms application. The Razor View Engine uses @ to render server-side content. The ASPX/web form view engine uses "<%= %>" or "<%: %>" to render server-side content. By default all text from an @ expression is HTML encoded. There is a different syntax ("<%: %>") to make text HTML encoded. Razor does not require the code block to be closed, the Razor View Engine parses itself and it is able to decide at runtime which is a content element and which is a code element. A web form view engine requires the code block to be closed properly otherwise it throws a runtime exception. The Razor View Engine prevents Cross Site Scripting (XSS) attacks by encoding the script or HTML tags before rendering to the view. A web form View engine does not prevent Cross Site Scripting (XSS) attack. The Razor Engine supports Test Driven Development (TDD). Web Form view engine does not support Test Driven Development (TDD) because it depends on the System.Web.UI.Page class to make the testing complex. Razor uses "@* â€¦ *@" for multiline comments. The ASPX View Engine uses "" for markup and "/ â€¦ */" for C# code. There are only three transition characters with the Razor View Engine. There are only three transition characters with the Razor View Engine.

The Razor View Engine is bit slower than the ASPX View Engine. Razor provides a new view engine with streamlined code for focused templating. Razor's syntax is very compact and improves readability of the markup and code. By default MVC supports ASPX (web forms) and Razor View Engine. MVC also supports third-party view engines like Spark, Nhaml, NDjango, SharpDOM and so on. ASP.NET MVC is open source.

What are the Main Razor Syntax Rules?

There are following types of Razor syntax-

• Razor code blocks are enclosed in @{ ... }

• Inline expressions (variables and functions) start with @

• Code statements end with semicolon

• Variables are declared with the var keyword

• Strings are enclosed with quotation marks

• C# code is case sensitive

• C# files have the extension .cshtml

C# Example:

@ { varmyMessage = "Hello World"; }

```
<!-- Inline expression or variable -->  < p > The value of
              myMessage is: @myMessage < /p>
              <!-- Multi-statement block -->
      @ {     var greeting = "Welcome to our site!";
           varweekDay = DateTime.Now.DayOfWeek;
 vargreetingMessage = greeting + " Here in Huston it is: " +
                     weekDay;        }
       < p > The greeting is: @greetingMessage < /p>
```

How do you implement Forms authentication in MVC?

Authentication is giving access to the user for a specific service by verifying his/her identity using his/her credentials like username and password or email and password. It assures that the correct user is authenticated or logged in for a specific service and the right service has been provided to the specific user based on their role that is nothing but authorization.

ASP.NET forms authentication occurs after IIS authentication is completed. You can configure forms authentication by using forms element with in web.config file of your application. The default attribute values for forms authentication are shown below:

<system.web> <authenticationmode="Forms">

<formsloginUrl="Login.aspx" protection="All" timeout="30" name=".ASPXAUTH" path="/" requireSSL="false" slidingExpiration="true" defaultUrl="default.aspx" cookieless="UseDeviceProfile" enableCrossAppRedirects="false" />

</system.web>

The FormsAuthentication class creates the authentication cookie automatically when SetAuthCookie() or RedirectFromLoginPage() methods are called. The value of authentication cookie contains a string representation of the encrypted and signed FormsAuthenticationTicket object.

Explain Areas in MVC?

From ASP.Net MVC 2.0 Microsoft provided a new feature in MVC applications, Areas. Areas are just a way to divide or “isolate” the modules of large applications in multiple or separated MVC.

When you add an area to a project, a route for the area is defined in an AreaRegistration file. The route sends requests to the area based on the request URL. To register routes for areas, you add code to theGlobal.asax file that can automatically find the area routes in the AreaRegistration file. AreaRegistration.RegisterAllAreas(); Benefits of Area in MVC: Allows us to organize models, views and controllers into separate functional sections of the application, such as administration, billing, customer support and much more. Easy to integrate with other Areas created by another. Easy for unit testing.

Explain the need of display mode in MVC?

DisplayModes give you another level of flexibility on top of the default capabilities we saw in the last section. DisplayModes can also be used along with the previous feature so we will simply build off of the site we just created.

Using display modes involves in 2 steps:

We should register Display Mode with a suffix for particular browser using "DefaultDisplayMode"e class inApplication_Start() method in the Global.asax file.

View name for particular browser should be appended with suffix mentioned in first step.

1. Desktop browsers (without any suffix. e.g.: Index.cshtml, _Layout.cshtml).
2. Mobile browsers (with a suffix "Mobile". e.g.:

Index.Mobile.cshtml,Layout.Mobile.cshtml)

If you want design different pages for different mobile device browsers (any different browsers) and render them depending on the browser requesting. To handle these requests you can register custom display modes. We can do that using

DisplayModeProvider.Instance.Modes.Insert(int index, IDisplayMode item) method.

Explain the concept of MVC Scaffolding?

ASP.NET Scaffolding is a code generation framework for ASP.NET Web applications. Visual Studio 2013 includes pre-installed code generators for MVC and Web API projects. You add scaffolding to your project when you want to quickly add code that interacts with data models. Using scaffolding can reduce the amount of time to develop standard data operations in your project.

Scaffolding consists of page templates, entity page templates, field page templates, and filter templates. These templates are called Scaffold templates and allow you to quickly build a functional data-driven Website.

Scaffolding Templates:

Create: It creates a View that helps in creating a new record for the Model. It automatically generates a label and input field for each property in the Model.

Delete: It creates a list of records from the model collection along with the delete link with delete record.

Details: It generates a view that displays the label and an input field of the each property of the Model in the MVC framework.

Edit: It creates a View with a form that helps in editing the current Model. It also generates a form with label and field for each property of the model.

List: It generally creates a View with the help of a HTML table that lists the Models from the Model Collection. It also generates a HTML table column for each property of the Model.

What is Route Constraints in MVC?

Routing is a great feature of MVC, it provides a REST based URL that is very easy to remember and improves page ranking in search engines.

This article is not an introduction to Routing in MVC, but we will learn a few features of routing and by implementing them we can develop a very flexible and user-friendly application. So, let's start without wasting valuable time.

start without wasting valuable time.

Add constraint to URL:

This is very necessary for when we want to add a specific constraint to our URL. Say, for example we want a URL. So, we want to set some constraint string after our host name. Fine, let's see how to implement it.

It's very simple to implement, just open the RouteConfig.cs file and you will find the routing definition in that. And modify the routing entry as in the following. We will see that we have added “abc” before.

Controller name, now when we browse we need to specify the string in the URL, as in the following:

What is Output Caching in MVC?

The main purpose of using Output Caching is to dramatically improve the performance of an ASP.NET MVC Application. It enables us to cache the content returned by any controller method so that the same content does not need to be generated each time the same controller method is invoked. Output Caching has huge advantages, such as it reduces server round trips, reduces database server round trips, reduces network traffic etc.

Keep the following in mind:

• Avoid caching contents that are unique per user.

• Avoid caching contents that are accessed rarely.

• Use caching for contents that are accessed frequently.

My MVC application displays a list of database records on the view page so by default each time the user invokes the controller method to see records, the application loops through the entire process and executes the database query. And this can actually decrease the application performance. So, we can advantage of the "Output Caching" that avoids executing database queries each time the user invokes the controller method. Here the view page is retrieved from the cache instead of invoking the controller method and doing redundant work.

Cached Content Locations:

In the above paragraph I said, in Output Caching the view page is retrieved from the cache, so where is the content cached/stored? Please note, there is no guarantee that content will be cached for the amount of time that we specify. When memory resources become low, the cache starts evicting content automatically.

OutputCache label has a "Location" attribute and it is fully controllable. Its default value is "Any", however there are the following locations available; as of now, we can use any one.

1. Any
2. Client
3. Downstream
4. Server
5. None
6. ServerAndClient

With "Any", the output cache is stored on the server where the request was processed. The recommended store cache is always on the server very carefully. You will learn about some security related tips in the following "Don't use Output Cache".

What is Bundling and Minification in MVC?

Bundling and minification are two new techniques introduced to improve request load time. It improves load time by reducing the number of requests to the server and reducing the size of requested assets (such as CSS and JavaScript).

Bundling: It lets us combine multiple JavaScript (.js) files or multiple cascading style sheet (.css) files so that they can be downloaded as a unit, rather than making individual HTTP requests.

Minification: It squeezes out whitespace and performs other types of compression to make the downloaded files as small as possible. At runtime, the process identifies the user agent, for example IE, Mozilla, etc. and then removes whatever is specific to Mozilla when the request comes from IE.

What is Validation Summary in MVC?

The ValidationSummary helper method generates an unordered list (ul element) of validation messages that are in the ModelStateDictionary object.

The ValidationSummary can be used to display all the error messages for all the fields. It can also be used to display custom error messages. The following figure shows how ValidationSummary displays the error messages.

ValidationSummary() Signature:

MvcHtmlStringValidateMessage(bool excludePropertyErrors, string message, object htmlAttributes)

Display field level error messages using ValidationSummary: By default, ValidationSummary filters out field level error messages. If you want to display field level error messages as a summary then specify excludePropertyErrors = false.

Example: ValidationSummary to display field errors: @Html.ValidationSummary(false, "", new { @class = "text-danger" }) So now, the following Edit view will display error messages as a summary at the top. Please make sure that you don't have a ValidationMessageFor method for each of the fields.

What is Database First Approach in MVC using Entity Framework?

Database First Approach is an alternative to the Code First and Model First approaches to the Entity Data Model which creates model codes (classes, properties, DbContextetc) from the database in the project and that class behaves as the link between database and controller.

There are the following approachs which is used to connect with database to application.

• Database First

• Model First

• Code First

Database first is nothing but only an approach to create web application where database is available first and can interact with the database. In this database, database is created first and after that we manage the code. The Entity Framework is able to generate a business model based on the tables and columns in a relational database.

What are the Folders in MVC application solutions?

When you create a project a folder structure gets created by default under the name of your project which can be seen in solution explorer. Below i will give you a brief explanation of what

project which can be seen in solution explorer. Below I will give you a brief explanation of what these folders are for.

Model: This folder contains classes that are used to provide data. These classes can contain data that is retrived from the database or data inserted in the form by the user to update the database.

Controllers: These are the classes which will perform the action invoked by the user. These classes contain methods known as "Actions" which responds to the user action accordingly.

Views: These are simple pages which use the model class data to populate the HTML controls and render it to the client browser. App_Start: Contains Classes such as FilterConfig, RoutesConfig, WebApiConfig. As of now we need to understand the RouteConfig class. This class contains the default format of the url that should be supplied in the browser to navigate to a specified page.

What are the methods of handling an Error in MVC?

Exception handling may be required in any application, whether it is a web application or a Windows Forms application. ASP.Net MVC has an attribute called "HandleError" that provides built-in exception filters. The HandleError attribute in ASP.NET MVC can be applied over the action method as well as Controller or at the global level. The HandleError attribute is the default implementation of IExceptionFilter. When we create a MVC application, the HandleError attribute is added within the Global.asax.cs file and registered in the Application_Start event.

public static void RegisterGlobalFilters(GlobalFilterCollection filters) { filters.Add(new HandleErrorAttribute()); }

protected void Application_Start() { AreaRegistration.RegisterAllAreas(); RegisterGlobalFilters(GlobalFilters.Filters); RegisterRoutes(RouteTable.Routes); }

Important properties of HandleError attribute: The HandleError Error attribute has a couple for properties that are very useful in handling the exception.

ExceptionType: Type of exception to be catch. If this property is not specified then the HandleError filter handles all exceptions.

View: Name of the view page for displaying the exception information.

Master: Master View for displaying the exception.

Order: Order in which the action filters are executed. The Order property has an integer value and it specifies the priority from 1 to any positive integer value. 1 means highest priority and the greater the value of the integer is, the lower is the priority of the filter.

AllowMultiple: It indicates whether more than one instance of the error filter attribute can be specified. Example:

[HandleError(View = "Error")]

public class HomeController: Controller { public ActionResult Index() { ViewBag.Message = "Welcome to ASP.NET MVC!"; int u = Convert.ToInt32(""); // Error line return View(); } }

HandleError Attribute at Action Method Level,

[HandleError(View = "Error")] public ActionResult Index() { ViewBag.Message = "Welcome to ASP.NET MVC!"; int u = Convert.ToInt32(""); // Error line return View(); }

What is Scaffolding in MVC?

Scaffolding is a code generation framework for ASP.NET Web applications. Visual Studio 2013 includes pre-installed code generators for MVC and Web API projects. You add scaffolding to your project when you want to quickly add code that interacts with data models. Using

scaffolding can reduce the amount of time to develop standard data operations in your project.

Prerequisites: To use ASP.NET Scaffolding, you must have:

• Microsoft Visual Studio 2013

• Web Developer Tools (part of default Visual Studio 2013 installation)

• ASP.NET Web Frameworks and Tools 2013 (part of default Visual Studio 2013 installation)

Advantages of using Scaffolding:

Minimal or no code to create a data-driven Web applications.

Quick development time.

Pages that are fully functional and include display, insert, edit, delete, sorting, and paging functionalities.

Built-in data validation that is based on the database schema.

Filters that are created for each foreign key or Boolean fields.

What is ViewStart?

Razor View Engine introduced a new layout named _ViewStart which is applied on all view automatically. Razor View Engine firstly executes the _ViewStart and then start rendering the other view and merges them.

Example of Viewstart: @ { Layout = "~/Views/Shared/_v1.cshtml"; } < !DOCTYPE html > < html > < head > < meta name = "viewport" content = "width=device-width" / > < title > ViewStart < /title> < /head> < body > ….. < /body> < /html>

What is JsonResultType in MVC?

Action methods on controllers return JsonResult (JavaScript Object Notation result) that can be used in an AJAX application. This class is inherited from the "ActionResult" abstract class. Here Json is provided one argument which must be serializable. The JSON result object that serializes the specified object to JSON format.

Example: public JsonResult JsonResultTest() { Return Json("Hello My Friend!"); }

What is TempData?

Tempdata-

TempData is a dictionary object derived from the TempDataDictionary class.

TempData is used to pass data from the current request to a subsequent request, in other words in the case of redirection.

The life of a TempData is very short and it retains its value for a short period of time.

It requires typecasting for complex data type as I've used in my example:

@foreach (var item in (List<MVCSample.Models.EmpRegistration>)TempData["EmployeeRegistration"])

You can retain its value using the Keep method for subsequent requests.

How to use ViewBag?

ViewBag is dynamic property that takes advantage of new dynamic features in C# 4.0. It's also

ViewBag is dynamic property that takes advantage of new dynamic features in C# 4.0. It's also used to pass data from a controller to a view. In short, The ViewBag property is simply a wrapper around the ViewData that exposes the ViewData dictionary as a dynamic object. Now create an action method "StudentSummary" in the "DisplayDataController" controller that stores a Student class object in ViewBag.

public ActionResult StudentSummary() { var student = new Student() { Name = "Sandeep Singh Shekhawat", Age = 24, City = "Jaipur" }; ViewBag.Student = student; return View(); } Thereafter create a view StudentSummary ("StudentSummary.cshtml") that shows student object data. ViewBag does not require typecasting for complex data type so you can directly access the data from ViewBag. @ { ViewBag.Title = "Student Summary"; var student = ViewBag.Student; } < table >< tr > < th > Name < /th> < th > Age < /th> < th > City < /th> < /tr> < tr > < td > @student.Name < /td> < td > @student.Age < /td> < td > @student.City < /td> < /tr> < /table> Here we used one more thing, "ViewBag.Title", that shows the title of the page.

What is Data Annotation Validator Attributes in MVC?

DataAnnotation plays a vital role in added validation to properties while designing the model itself. This validation can be added for both the client side and the server side.You understand that decorating the properties in a model with an Attribute can make that property eligible for Validation.

Some of the DataAnnotation used for validation are given below:

Required Specify a property as required. [Required(ErrorMessage="CustomerName is mandatory")]

RegularExpression Specifies the regular expression to validate the value of the property. [RegularExpression("[a-z]", ErrorMessage = "Invalid character")]

Range Specifies the Range of values between which the property values are checked. [Range(1000,10000,ErrorMessage="Range should be between 1k & 10k")]

StringLength Specifies the Min & Max length for a string property. [StringLength(50, MinimumLength = 5, ErrorMessage = "Minimum char is 5 and maximum char is 10")]

MaxLength Specifies the Max length for the property value. [MaxLength(10,ErrorMessage="Customer Code is exceeding")]

MinLength It is used to check for minimum length. [MinLength(5, ErrorMessage = "Customer Code is too small")]

How can we done Custom Error Page in MVC?

The HandleErrorAttribute allows you to use a custom page for this error. First you need to update your web.config file to allow your application to handle custom errors.

<system.web> </system.web>

Then, your action method needs to be marked with the atttribute.

[HandleError] public class HomeController: Controller { [HandleError] publicActionResultThrowException() { throw new ApplicationException(); } }

By calling the ThrowException action, this would then redirect the user to the default error page. In our case though, we want to use a custom error page and redirect the user there instead.So, let's create our new custom view page.

Next, we simply need to update the HandleErrorAttribute on the action method.

[HandleError] public class HomeController: Controller { [HandleError(View = "CustomErrorView")] publicActionResultThrowException() { throw new ApplicationException(); } }

}

Server Side Validation in MVC?

The ASP.NET MVC Framework validates any data passed to the controller action that is executing, it populates a ModelState object with any validation failures that it finds and passes that object to the controller. Then the controller actions can query the ModelState to discover whether the request is valid and react accordingly.

I will use two approaches in this article to validate a model data. One is to manually add an error to the ModelState object and another uses the Data Annotation API to validate the model data.

Approach 1: Manually Add Error to ModelState object-

I create a User class under the Models folder. The User class has two properties "Name" and "Email". The "Name" field has required field validations while the "Email" field has Email validation. So let's see the procedure to implement the validation. Create the User

Model as in the following:

namespace ServerValidation.Models { public class User { public string Name {get; set; } public string Email { get; set; } } }

After that I create a controller action in User Controller (UserController.cs under Controllers folder). That action method has logic for the required validation for Name and Email validation on the Email field. I add an error message on ModelState with a key and that message will be shown on the view whenever the data is not to be validated in the model.

using System.Text.RegularExpressions; using System.Web.Mvc; namespace ServerValidation.Controllers { public class UserController: Controller { public ActionResult Index() { return View(); }

[HttpPost] public ActionResult Index(ServerValidation.Models.User model) { if (string.IsNullOrEmpty(model.Name)) { ModelState.AddModelError("Name", "Name is required"); } if (!string.IsNullOrEmpty(model.Email))

{ string emailRegex = @ "^([a-zA-Z0-9_-.]+)@(([[0-9]{1,3}" + @ ".[0- 9]{1,3}.[0-9]{1,3}.)|(([a-zA-Z0-9-]+" + @ ".)+))([a-zA-Z]{2,4}|[0-9]{1,3})(]?)$"; Regex re = new Regex(emailRegex);

if (!re.IsMatch(model.Email))

{ ModelState.AddModelError("Email", "Email is not valid"); }

} else { ModelState.AddModelError("Email", "Email is required"); }

if (ModelState.IsValid)

{ ViewBag.Name = model.Name; ViewBag.Email = model.Email; }

return View(model); } } }

Thereafter I create a view (Index.cshtml) for the user input under the User folder.

@model ServerValidation.Models.User

@ { ViewBag.Title = "Index"; }

@using(Html.BeginForm())

{ if (@ViewData.ModelState.IsValid)

{ if (@ViewBag.Name != null)

{ < b > Name: @ViewBag.Name < br / > Email: @ViewBag.Email < /b> }

} < fieldset > < legend > User < /legend> < div class = "editor-label" >

@Html.LabelFor(model => model.Name) < /div> < div class = "editor-field" >

@Html.EditorFor(model => model.Name)

@if(!ViewData.ModelState.IsValid)

{< span class = "field-validation-error" > @ViewData.ModelState["Name"].Errors[0].ErrorMessage < /span> } < /div> < div class = "editor-label" > @Html.LabelFor(model => model.Email) < /div> < div class = "editor-field" > @Html.EditorFor(model => model.Email) @if(!ViewData.ModelState.IsValid) { < span class = "field-validation-error" > @ViewData.ModelState["Email"].Errors[0].ErrorMessage < /span> } < /div> < p > < input type = "submit" value = "Create" / > < /p> < /fieldset> }

What is the use of remote validation in MVC?

Remote validation is the process where we validate specific data posting data to a server without posting the entire form data to the server. Let's see an actual scenario, in one of my projects I had a requirement to validate an email address, whetehr it already exists in the database. Remote validation was useful for that; without posting all the data we can validate only the email address supplied by the user.

Practical Explanation

Let's create a MVC project and name it accordingly, for me its “TestingRemoteValidation”. Once the project is created let's create a model named UserModel that will look like:

public class UserModel { [Required] public string UserName {get;set;}

[Remote("CheckExistingEmail", "Home", ErrorMessage = "Email already exists!")] public string UserEmailAddress{get; set;} }

Let's get some understanding of the remote attribute used, so the very first parameter “CheckExistingEmail” is the the name of the action. The second parameter “Home” is referred to as controller so to validate the input for the UserEmailAddress the “CheckExistingEmail” action of the “Home” controller is called and the third parameter is the error message. Let's implement the “CheckExistingEmail” action result in our home controller.

public ActionResult CheckExistingEmail(string UserEmailAddress) { bool ifEmailExist = false; try { ifEmailExist = UserEmailAddress.Equals("mukeshknayak@gmail.com") ? true : false; return Json(!ifEmailExist, JsonRequestBehavior.AllowGet); } catch (Exception ex){return Json(false, JsonRequestBehavior.AllowGet); } }

What are the Exception filters in MVC?

Exception is part and parcel of an application. They are a boon and a ban for an application too. Isn't it? This would be controversial, for developers it helps them track minor and major defects in an application and sometimes they are frustrating when it lets users land on the Yellow screen of death each time. This would make the users mundane to the application. Thus to avoid this, developers handle the exceptions. But still sometimes there are a few unhandled exceptions.

Now what is to be done for them? MVC provides us with built-in "Exception Filters" about which we will explain here.

Get Started:

Exception filters run when some of the exceptions are unhandled and thrown from an invoked action. The reason for the exception can be anything and so is the source of the exception.

Creating an Exception Filter:

Custom Exception Filters must implement the builtinIExceptionFilter interface. The interface looks as in the following:

public interface IExceptionFilter { void OnException(ExceptionContext filterContext) }

Whenever an unhandled exception is encountered, the OnException method gets invoked. The parameter as we can see, ExceptionContext is derived from the ControllerContext and has a number of built-in properties that can be used to get the information about the request causing the exception. Their property's ExceptionContextpassess are shown in the following table:

Name

Type

Detail

Result

ActionResult

The result returned by the action being invoked.

Exception

The unhandled exceptions caused from the actions in the applications.

ExceptionHandled

BOOL This is a very handy property that returns a bool value (true/false) based on if the exception is handled by any of the filters in the applicaiton or not.

The exception being thrown from the action is detailed by the Exception property and once handled (if), and then the property ExceptionHandled can be toggled, so that the other filters would know if the exception has been already handled and cancel the other filter requests to handle. The problem is that if the exceptions are not handled, then the default MVC behavior shows the dreaded yellow screen of death. To the users, that makes a very impression on the users and more importantly, it exposes the application's handy and secure information to the outside world that may have hackers and then the application gets into the road to hell. Thus, the exceptions need to be dealt with very carefully. Let's show one small custom exception filter. This filter can be stored inside the Filters folder in the web project of the solution. Let's add a file/class called CustomExceptionFilter.cs.

public class CustomExceptionFilter: FilterAttribute, IExceptionFilter { public void OnException(ExceptionContext filterContext). { if (!filterContext.ExceptionHandled && filterContext.Exception is NullReference Exception) { filterContext.Result = new RedirectResult("customErrorPage.html"); filterContext.ExceptionHandled = true; } } }

What is MVC HTML- Helpers and it’s Methods?

Helper methods are used to render HTML in the view. Helper methods generate HTML output that is part of the view. They provide an advantage over using the HTML elements since they can be reused across the views and also requires less coding. There are several builtin helper methods that are used to generate the HTML for some commonly used HTML elements, like form, checkbox, dropdownlist etc. Also we can create our own helper methods to generate custom HTML. First we will see how to use the builtin helper methods and then we will see how to create custom helper methods.

Standard HtmlHelper methods: Some of the standard helper methods are:

• ActionLink: Renders an anchor

• ActionLink: Renders an anchor.

• BeginForm: Renders HTML form tag

• CheckBox: Renders check box.

• DropDownList: Renders drop-down list.

• Hidden: Renders hidden field

• ListBox: Renders list box.

• Password: Renders TextBox for password input

• RadioButton: Renders radio button.

• TextArea: Renders text area.

• TextBox: Renders text box.

Define Controller in MVC?

The controller provides model data to the view, and interprets user actions such as button clicks. The controller depends on the view and the model. In some cases, the controller and the view are the same object.

The Controllers Folder: The Controllers Folder contains the controller classes responsible for handling user input and responses. MVC requires the name of all controllers to end with "Controller".

In our example, Visual Web Developer has created the following files: HomeController.cs (for the Home and about pages) and AccountController.cs (For the Log On pages):

Explain Model in MVC?

The model represents the data, and does nothing else. The model does NOT depend on the controller or the view. The MVC Model contains all application logic (business logic, validation logic, and data access logic), except pure view and controller logic. With MVC, models both hold and manipulate application data.

The Models Folder: The Models Folder contains the classes that represent the application model. Visual Web Developer automatically creates an AccountModels.cs file that contains the models for application security.

Explain View in MVC?

A view is responsible for displaying all of, or a portion of, data for users. In simple terms, whatever we see on the output screen is a view.

The Views Folder: The Views folder stores the files (HTML files) related to the display of the application (the user interfaces). These files may have the extensions html, asp, aspx, cshtml, and vbhtml, depending on the language content.

The Views folder contains one folder for each controller. Visual Web Developer has created an Account folder, a Home folder, and a Shared folder (inside the Views folder). The Account folder contains pages for registering and logging in to user accounts. The Home folder is used for storing application pages like the home page and the about page. The Shared folder is used to store views shared between controllers (master pages and layout pages).

What is Attribute Routing in MVC?

A route attribute is defined on top of an action method. The following is the example of a Route Attribute in which routing is defined where the action method is defined.

Attribute in which routing is defined where the action method is defined.

In the following example, I am defining the route attribute on top of the action method:

public class HomeController: Controller { //URL: /Mvctest [Route(“Mvctest”)] public ActionResult Index(){ ViewBag.Message = "Welcome to ASP.NET MVC!"; return View(); } }

Attribute Routing with Optional Parameter-

We can also define an optional parameter in the URL pattern by defining a question mark (“?") to the route parameter. We can also define the default value by using parameter=value.

public class HomeController: Controller { // Optional URI Parameter // URL: /Mvctest/ // URL: /Mvctest/0023654

[Route(“Mvctest /{ customerName ? }”)] public ActionResult OtherTest(string customerName) ViewBag.Message = "Welcome to ASP.NET MVC!"; return View(); } // Optional URI Parameter with default value // URL: /Mvctest/ // URL: /Mvctest/0023654

[Route(“Mvctest /{ customerName = 0036952}”)] public ActionResult OtherTest(string customerName) { ViewBag.Message = "Welcome to ASP.NET MVC!"; return View(); } }

Explain RenderSection in MVC?

RenderSection() is a method of the WebPageBase class. Scott wrote at one point, The first parameter to the "RenderSection()" helper method specifies the name of the section we want to render at that location in the layout template. The second parameter is optional, and allows us to define whether the section we are rendering is required or not. If a section is "required", then Razor will throw an error at runtime if that section is not implemented within a view template that is based on the layout file (that can make it easier to track down content errors). It returns the HTML content to render.

@RenderSection("featured", required: false)
@RenderBody()

What is GET and POST Actions Types?

GET - GET is used to request data from a specified resource. With all the GET request we pass the URL which is compulsory, however it can take the following overloads.

.get(url [, data] [, success(data, textStatus, jqXHR)] [, dataType]).done/.fail

POST - POST is used to submit data to be processed to a specified resource. With all the POST requests we pass the URL which is compulsory and the data, however it can take the following overloads.

.post(url [, data] [, success(data, textStatus, jqXHR)] [, dataType])

What's new in MVC 6?

In MVC 6 Microsoft removed the dependency of System.Web.Dll from MVC6 because it's so expensive that typically it consumes 30k of memory per request and response, whereas now MVC 6 only requires 2k of memory per request and the response is a very small memory consumtion.

The advantage of using the cloud-optimized framework is that we can include a copy of the mono CLR with your website. For the sake of one website we do not need to upgrade the .NET version on the entire machine. A different version of the CLR for a different website running side by side.

MVC 6 is a part of ASP.NET 5 that has been designed for cloud-optimized applications. The runtime automatically picks the correct version of the library when our MVC application is deployed to the cloud.

The Core CLR is also supposed to be tuned with a high resource-efficient optimization. Microsoft has made many MVC, Web API, WebPage and SignalLrpeices we call MVC 6.

Most of the problems are solved using the Roslyn Compiler. In ASP.NET vNext uses the Roslyn Compiler. Using the Roslyn Compiler we do not need to compile the application, it automatically compiles the application code. You will edit a code file and can then see the changes by refreshing the browser without stopping or rebuilding the project.

Run on hosts other than IIS:

Where we use MVC5 we can host it on an IIS server and we can also run it on top of an ASP.NET Pipeline, on the other hand MVC 6 has a feature that makes it better and that feature is itself hosted on an IIS server and a self-user pipeline.

Environment based configuration system:

The configuration system provides an environment to easily deploy the application on the cloud. Our application works just like a configuration provider. It helps to retrieve the value from the various configuration sources like XML file.

MVC 6 includes a new environment-based configuration system. Unlike something else it depends on just the Web.Config file in the previous version.

Dependency injection:

Using the IServiceProvider interface we can easily add our own dependency injection container. We can replace the default implementation with our own container.

Supports OWIN:

We have complete control over the composable pipeline in MVC 6 applications. MVC 6 supports the OWIN abstraction.

What is Domain Driven Design and Development?

Domain-Driven Design (DDD) is a collection of principles and patterns that help developers to take design decisions to develop elegant systems for different domains. It is not a technology or methodology.

The main components of DDD are: Entity, Value Object, Aggregate, Service and Repository.

Entity- An object that has an identity- it is unique within the system, like Customer, Employee etc.

Value Object- An object that has no identity within the system like Rate, State etc.

Note: A value object can become an entity depending on the situation.

Aggregate: An aggregate root is a special kind of entity that consumers refer to directly. All consumers of the aggregate root are called as aggregate. The aggregate root guarantees the consistency of changes being made within the aggregate.

Service- A service is a way of dealing with actions, operations and activities within your application.

Repository- A repository is responsible to store and to retrieve your data. It is not a concern how and where data will be persist. So, it can be SQL server, oracle, xml, text file or anything else. Repository is not a Data Access Layer but it refers to a location for storage, often for safety or preservation.

For more info refer this link http://msdn.microsoft.com/en-us/magazine/dd419654.aspx

What is MVP pattern?

This pattern is similar to MVC pattern in which controller has been replaced by the presenter. This design pattern splits an application into three main aspects: Model, View and Presenter.

Model - The Model represents a set of classes that describes the business logic and data. It also defines business rules for data means how the data can be changed and manipulated.

View - The View represents the UI components like CSS, jQuery, html etc. It is only responsible for displaying the data that is received from the presenter as the result. This also transforms the model(s) into UI.

Presenter - The Presenter is responsible for handling all UI events on behalf of the view. This receive input from users via the View, then process the user's data with the help of Model and passing the results back to the View.

Unlike view and controller, view and presenter are completely decoupled from each other’s and communicate to each other’s by an interface.

Also, presenter does not manage the incoming request traffic as controller.

This pattern is commonly used with ASP.NET Web Forms applications which require to create automated unit tests for their code-behind pages. This is also used with windows forms.

Key Points about MVP Pattern

User interacts with the View.

There is one-to-one relationship between View and Presenter means one View is mapped to only one Presenter.

View has a reference to Presenter but View has not reference to Model.

Provides two way communication between View and Presenter.

What is MVVM pattern?

MVVM stands for Model-View-View Model. This pattern supports two-way data binding between view and View model. This enables automatic propagation of changes, within the state of view model to the View. Typically, the view model uses the observer pattern to notify changes in the view model to model.

Model - The Model represents a set of classes that describes the business logic and data. It also defines business rules for data means how the data can be changed and manipulated.

View - The View represents the UI components like CSS, jQuery, html etc. It is only responsible for displaying the data that is received from the controller as the result. This also transforms the model(s) into UI.

View Model - The View Model is responsible for exposing methods, commands, and other properties that helps to maintain the state of the view, manipulate the model as the result of actions on the view, and trigger events in the view itself.

This pattern is commonly used by the WPF, Silverlight, Caliburn, nRoute etc.

Key Points about MVVM Pattern

User interacts with the View.

There is many-to-one relationship between View and ViewModel means many View can be mapped to one ViewModel.

View has a reference to ViewModel but View Model has no information about the View.

Supports two-way data binding between View and ViewModel.

How MVC pattern works in ASP.NET MVC?

Working of MVC pattern in ASP.NET MVC is explained as below:

The Model in ASP.NET MVC

The Model in ASP.NET MVC can be broken down into several different layers as given below: Objects or ViewModel or Presentation Layer - This layer contains simple objects or complex objects which are used to specify strongly-typed view. These objects are used to pass data from controller to strongly-typed view and vice versa. The classes for these objects can have specific validation rules which are defined by using data annotations. Typically, these classes have those properties which you want to display on corresponding view/page.

Business Layer - This layer helps you to implement your business logic and validations for your application. This layer make use of Data Access Layer for persisting data into database. Also, this layer is directly invoked by the Controller to do processing on input data and sent back to view.

Data Access Layer - This layer provides objects to access and manipulate the database of your application. Typically, this layer is made by using ORM tools like Entity Framework or NHibernate etc.

By default, models are stored in the Models folder of an ASP.NET MVC application.

The View in ASP.NET MVC

The view is only responsible for displaying the data that is received from the controller as a result. It also responsible for transforming a model or models into UI which provide all the required business logic and validation to the view. By default, views are stored in the Views folder of an ASP.NET MVC application.

The Controller in ASP.NET MVC

The Controller in ASP.NET MVC, respond to HTTP requests and determine the action to take based upon the content of the incoming request. It receives input from users via the View, then process the user's data with the help of Model and passing the results back to the View.

By default, controllers are stored in the Controllers folder an ASP.NET MVC application.

Explain brief history of ASP.NET MVC?

Here is the list of released version history of ASP.NET MVC Framework with theirs features.

ASP.NET MVC1

Released on Mar 13, 2009 Runs on .NET 3.5 and with Visual Studio 2008 & Visual Studio 2008 SP1 MVC Pattern architecture with WebForm Engine Html Helpers Ajax helpers Routing Unit Testing

ASP.NET MVC2

Released on Mar 10, 2010 Runs on .NET 3.5, 4.0 and with Visual Studio 2008 & 2010 Strongly typed HTML helpers means lambda expression based Html Helpers Templated Helpers UI helpers with automatic scaffolding & customizable templates Support for DataAnnotations Attributes to apply model validation on both client and server sides Overriding the HTTP Method Verb including GET, PUT, POST, and DELETE Areas for partitioning a large applications into modules Asynchronous controllers

ASP.NET MVC3

Released on Jan 13, 2011 Runs on .NET 4.0 and with Visual Studio 2010 The Razor view engine Enhanced Data Annotations attributes for model validation on both client and server sides Remote Validation Compare Attribute Session less Controller Child Action Output Caching Dependency Resolver Entity Framework Code First support Partial-page output caching ViewBag dynamic property for passing data from controller to view Global Action Filters Better JavaScript support with unobtrusive JavaScript, jQuery Validation, and JSON binding Use of NuGet to deliver software and manage dependencies throughout the platform

ASP.NET MVC4

Released on Aug 15, 2012 Runs on .NET 4.0, 4.5 and with Visual Studio 2010SP1 & Visual Studio 2012 ASP.NET WEB API Enhancements to default project templates Mobile project template using jQuery Mobile Display Modes Task support for Asynchronous Controllers Bundling and minification Support for the Windows Azure SDK

ASP.NET MVC5

Released on 17 October 2013 Runs on .NET 4.5, 4.5.1 and with Visual Studio 2012 & Visual Studio 2013 One ASP.NET ASP.NET Identity

ASP.NET Scaffolding

Authentication filters - run prior to authorization filters in the ASP.NET MVC pipeline Bootstrap in the MVC template ASP.NET WEB API2

What is difference between 3 layer architecture and MVC architecture?

3-layer architecture separates the application into 3 components which consists of Presentation Layer Business Layer and Data Access Layer. In 3-layer architecture, user interacts with the Presentation layer. 3-layer is a linear architecture.

MVC architecture separates the application into three components which consists of Model, View and Controller. In MVC architecture, user interacts with the controller with the help of view. MVC is a triangle architecture.

MVC does not replace 3-layer architecture. Typically 3-layer and MVC are used together and MVC acts as the Presentation layer.

What is ViewModel in ASP.NET MVC?

In ASP.NET MVC, ViewModel is a class that contains the fields which are represented in the strongly-typed view. It is used to pass data from controller to strongly-typed view.

Key Points about ViewModel

ViewModel contain fields that are represented in the view (for LabelFor, EditorFor, DisplayFor helpers)

ViewModel can have specific validation rules using data annotations.

ViewModel can have multiple entities or objects from different data models or data source.

Explain ASP.NET MVC pipeline?

The detail ASP.NET MVC pipeline is given below:

Routing - Routing is the first step in ASP.NET MVC pipeline. Typically, it is a pattern matching system that matches the incoming request to the registered URL patterns in the Route Table.

The UrlRoutingModule(System.Web.Routing.UrlRoutingModule) is a class which matches an incoming HTTP request to a registered route pattern in the RouteTable(System.Web.Routing.RouteTable).

RouteTable(System.Web.Routing.RouteTable).

Controller Initialization - The MvcHandler initiates the real processing inside ASP.NET MVC pipeline by using ProcessRequest method. This method uses the IControllerFactory instance (default is System.Web.Mvc.DefaultControllerFactory) to create corresponding controller.

Action Execution – Action execution occurs in the following steps:

When the controller is initialized, the controller calls its own InvokeAction() method by passing the details of the chosen action method. This is handled by the IActionInvoker.

After chosen of appropriate action method, model binders(default is System.Web.Mvc.DefaultModelBinder) retrieves the data from incoming HTTP request and do the data type conversion, data validation such as required or date format etc. and also take care of input values mapping to that action method parameters.

Authentication Filter was introduced with ASP.NET MVC5 that run prior to authorization filter. It is used to authenticate a user. Authentication filter process user credentials in the request and provide a corresponding principal. Prior to ASP.NET MVC5, you use authorization filter for authentication and authorization to a user. By default, Authenticate attribute is used to perform Authentication. You can easily create your own custom authentication filter by implementing IAuthenticationFilter.

Authorization filter allow you to perform authorization process for an authenticated user. For example, Role based authorization for users to access resources. By default, Authorize attribute is used to perform authorization. You can also make your own custom authorization filter by implementing IAuthorizationFilter.

Action filters are executed before (OnActionExecuting) and after (OnActionExecuted) an action is executed. IActionFilter interface provides you two methods OnActionExecuting and OnActionExecuted methods which will be executed before and after an action gets executed respectively. You can also make your own custom ActionFilters filter by implementing IActionFilter. For more about filters refer this article Understanding ASP.NET MVC Filters and Attributes

When action is executed, it process the user inputs with the help of model (Business Model or Data Model) and prepare Action Result.

Result Execution - Result execution occurs in the following steps:

Result filters are executed before (OnResultExecuting) and after (OnResultExecuted) the ActionResult is executed. IResultFilter interface provides you two methods OnResultExecuting and OnResultExecuted methods which will be executed before and after an ActionResult gets executed respectively. You can also make your own custom ResultFilters filter by implementing IResultFilter.

Action Result is prepared by performing operations on user inputs with the help of BAL or DAL. The Action Result type can be ViewResult, PartialViewResult, RedirectToRouteResult, RedirectResult, ContentResult, JsonResult, FileResult and EmptyResult.

Various Result type provided by the ASP.NET MVC can be categorized into two category- ViewResult type and NonViewResult type. The Result type which renders and returns an HTML page to the browser, falls into ViewResult category and other result type which returns only data either in text format, binary format or a JSON format, falls into NonViewResult category.

View Initialization and Rendering - View Initialization and Rendering execution occurs in the following steps:

ViewResult type i.e. view and partial view are represented by IView (System.Web.Mvc.IView) interface and rendered by the appropriate View Engine. This process is handled by IViewEngine (System.Web.Mvc.IViewEngine) interface of the view engine. By default ASP.NET MVC provides WebForm and Razor view engines. You can also create your custom engine by using IViewEngine interface and can registered your custom view engine in to your ASP.NET

using IViewEngine interface and can registered your custom view engine in to your ASP.NET MVC application as shown below:

Html Helpers are used to write input fields, create links based on the routes, AJAX-enabled forms, links and much more. Html Helpers are extension methods of the HtmlHelper class and can be further extended very easily. In more complex scenario, it might render a form with client side validation with the help of JavaScript or jQuery.

What are Routing in ASP.NET MVC?

Routing is a pattern matching system that monitor the incoming request and figure out what to do with that request. At runtime, Routing engine use the Route table for matching the incoming request's URL pattern against the URL patterns defined in the Route table. You can register one or more URL patterns to the Route table at Application_Start event.

When the routing engine finds a match in the route table for the incoming request's URL, it forwards the request to the appropriate controller and action. If there is no match in the route table for the incoming request's URL, it returns a 404 HTTP status code.

How to define a route in ASP.Net MVC?

You can define a route in ASP.NET MVC as given below:

public static void RegisterRoutes(RouteCollection routes) { routes.MapRoute("Default", // Route name "{controller}/{action}/{id}", // Route Pattern new { controller = "Home", action = "Index",id = UrlParameter.Optional }// Default values for above defined parameters);}

protected void Application_Start()

{RegisterRoutes(RouteTable.Routes); //TODO:}

Always remember route name should be unique across the entire application. Route name can't be duplicate.

In above example we have defined the Route Pattern {controller}/{action}/{id} and also provide the default values for controller, action and id parameters. Default values means if you will not provide the values for controller or action or id defined in the pattern then these values will be serve by the routing system.

Suppose your webapplication is running on www.example.com then the url pattren for you application will be www.example.com/{controller}/{action}/{id}. Hence you need to provide the controller name followed by action name and id if it is required. If you will not provide any of the value then default values of these parameters will be provided by the routing system. Here is a list of URLs that match and don't match this route pattern.

Request URL

Parameters http://example.com/ controller=Home, action=Index, id=none, Since default value of controller and action are Home and Index respectively.

http://example.com/Admin controller=Admin, action=Index, id=none, Since default value of action is Index

http://example.com/Admin/Product controller=Admin, action=Product, id=none

http://example.com/Admin/Product/1 controller=Admin, action=Product, id=1

http://example.com/Admin/Product/SubAdmin/1 No Match Found

http://example.com/Admin/Product/SubAdmin/Add/1 No Match Found

Note: Always put more specific route on the top order while defining the routes, since routing system check the incoming URL pattern form the top and as it get the matched route it will

consider that. It will not checked further routes after matching pattern.

What is Attribute Routing and how to define it?

ASP.NET MVC5 and WEB API 2 supports a new type of routing, called attribute routing. In this routing, attributes are used to define routes. Attribute routing provides you more control over the URIs by defining routes directly on actions and controllers in your ASP.NET MVC application and WEB API.

Controller level routing – You can define routes at controller level which apply to all actions within the controller unless a specific route is added to an action.

[RoutePrefix("MyHome")] [Route("{action=index}")] //default action public class HomeController : Controller {//new route: /MyHome/Index public ActionResult Index() {return View();} //new route: /MyHome/About public ActionResult About() {ViewBag.Message = "Your application description page."; return View();} //new route: /MyHome/Contact public ActionResult Contact() {ViewBag.Message = "Your contact page."; return View();} }

Action level routing – You can define routes at action level which apply to a specific action with in the controller.

public class HomeController : Controller {[Route("users/{id:int:min(100)}")] //route: /users/100 public ActionResult Index(int id) { //TO DO: return View();}

[Route("users/about")] //route" /users/about public ActionResult About() {ViewBag.Message = "Your application description page."; return View(); } //route: /Home/Contact public ActionResult Contact() {ViewBag.Message = "Your contact page."; return View();}}

Note:Attribute routing should configure before the convention-based routing. When you combine attribute routing with convention-based routing, actions which do not have Route attribute for defining attribute-based routing will work according to convention-based routing. In above example Contact action will work according to convention-based routing.

When you have only attribute routing, actions which do not have Route attribute for defining attribute-based routing will not be the part of attribute routing. In this way they can't be access from outside as a URI.

When to use Attribute Routing?

The convention-based routing is complex to support certain URI patterns that are common in RESTful APIs. But by using attribute routing you can define these URI patterns very easily.

For example, resources often contain child resources like Clients have orders, movies have actors, books have authors and so on. It's natural to create URIs that reflects these relations like as: /clients/1/orders

This type of URI is difficult to create using convention-based routing. Although it can be done, the results don't scale well if you have many controllers or resource types.

With attribute routing, it's pretty much easy to define a route for this URI. You simply add an attribute to the controller action as: [Route("clients/{clientId}/orders")] public IEnumerable GetOrdersByClient(int clientId) {//TO DO}

How to enable Attribute Routing in ASP.NET MVC?

Enabling attribute routing in your ASP.NET MVC5 application is simple, just add a call to routes.MapMvcAttributeRoutes() method with in RegisterRoutes() method of RouteConfig.cs file.

public class RouteConfig {public static void RegisterRoutes(RouteCollection routes) {routes.IgnoreRoute("{resource}.axd/{*pathInfo}"); //enabling attribute routing routes.MapMvcAttributeRoutes();}} You can also combine attribute routing with convention-

routes.MapMvcAttributeRoutes(); //You can also combine attribute routing with convention based routing. public class RouteConfig {public static void RegisterRoutes(RouteCollection routes) {routes.IgnoreRoute("{resource}.axd/{*pathInfo}"); //enabling attribute routing routes.MapMvcAttributeRoutes(); //convention-based routing routes.MapRoute(name: "Default", url: "{controller}/{action}/{id}", defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional });}}

How to define Attribute Routing for Area in ASP.NET MVC?

You can also define attribute routing for a controller that belongs to an area by using the RouteArea attribute. When you define attribute routing for all controllers with in an area, you can safely remove the AreaRegistration class for that area.

[RouteArea("Admin")] [RoutePrefix("menu")] [Route("{action}")] public class MenuController : Controller { // route: /admin/menu/login public ActionResult Login() {return View(); } // route: /admin/menu/products [Route("products")] public ActionResult GetProducts() {return View(); } // route: /categories [Route("~/categories")] public ActionResult Categories() { return View(); } }

What is difference between Routing and URL Rewriting?

Many developers compare routing to URL rewriting since both look similar and can be used to make SEO friendly URLs. But both the approaches are very much different. The main difference between routing and url rewriting is given below:

URL rewriting is focused on mapping one URL (new url) to another URL (old url) while routing is focused on mapping a URL to a resource.

URL rewriting rewrites your old url to new one while routing never rewrite your old url to new one but it map to the original route.

What is Route Constraints in ASP.NET MVC?

Route constraints is way to put some validation around the defined route.

Creating Route Constraints

Suppose we have defined the following route in our application and you want to restrict the incoming request url with numeric id only.Now let's see how to do it with the help of regular expression.

public static void RegisterRoutes(RouteCollection routes) {routes.MapRoute("Default", // Route name "{controller}/{action}/{id}", // Route Pattern new { controller = "Home", action = "Index", id = UrlParameter.Optional } // Default values for parameters);}

Restrict to numeric id only

public static void RegisterRoutes(RouteCollection routes) {routes.MapRoute("Default", // Route name "{controller}/{action}/{id}", // Route Pattern new {controller = "Home", action = "Index", id = UrlParameter.Optional }, // Default values for parameters new { id = @"\d+" } //Restriction for id);}

Now for this route, routing engine will consider only those URLs which have only numeric id like as http://example.com/Admin/Product/1 else it will considers that url is not matched with this route.

How route table is created in ASP.NET MVC?

When an MVC application first starts, the Application_Start() method in global.asax is called. This method calls the RegisterRoutes() method. The RegisterRoutes() method creates the route table for MVC application.

What are important namespaces in ASP.NET MVC?

What are important namespaces in ASP.NET MVC?

There are some important namespaces as given below:

System.Web.Mvc - This namespace contains classes and interfaces that support the MVC pattern for ASP.NET Web applications. This namespace includes classes that represent controllers, controller factories, action results, views, partial views, and model binders.

System.Web.Mvc.Ajax - This namespace contains classes that supports Ajax scripting in an ASP.NET MVC application. The namespace includes support for Ajax scripts and Ajax option settings as well.

System.Web.Mvc.Html – This namespace contains classes that help render HTML controls in an MVC application. This namespace includes classes that support forms, input controls, links, partial views, and validation.

What is difference View Engine?

A View Engine is a MVC subsystem which has its own markup syntax. It is responsible for converting server-side template into HTML markup and rendering it to the browser. Initially, ASP.NET MVC ships with one view engine, web forms (ASPX) and from ASP.NET MVC3 a new view engine, Razor is introduced. With ASP.NET MVC, you can also use other view engines like Spark, NHaml etc.

How View Engine works?

Each view engine has following three main components:

ViewEngine class - This class implements the IViewEngine interface and responsible for locating view templates.

View class - This class implements the IView interface and responsible for combining the template with data from the current context and convert it to output HTML markup.

Template parsing engine - This parses the template and compiles the view into executable code.

How to make Custom View Engine?

ASP.NET MVC is an open source and highly extensible framework. You can create your own View engine by Implementing IViewEngine interface or by inheriting VirtualPathProviderViewEngine abstract class.

public class CustomViewEngine : VirtualPathProviderViewEngine {

public CustomViewEngine() { // Define the location of the View and Partial View this.ViewLocationFormats = new string[] { "~~/Views/{1}/{0}.html", "~~/Views/Shared/{0}.html" };

this.PartialViewLocationFormats = new string[] { "~~/Views/{1}/{0}.html", "~~/Views/Shared/{0}.html" }; }

protected override IView CreatePartialView(ControllerContext controllerContext, string partialPath) { var physicalpath = controllerContext.HttpContext.Server.MapPath(partialPath); return new CustomView(physicalpath); }

protected override IView CreateView(ControllerContext controllerContext, string viewPath, string masterPath) { var physicalpath = controllerContext.HttpContext.Server.MapPath(viewPath); return new CustomView(physicalpath); } }

public class CustomView : IView { private string _viewPhysicalPath;

public CustomView(string ViewPhysicalPath) { _viewPhysicalPath = ViewPhysicalPath;}

```
public void Render(ViewContext viewContext, System.IO.TextWriter writer) { //Load File string rawcontents = File.ReadAllText(_viewPhysicalPath); //Perform Replacements string parsedcontents = Parse(rawcontents, viewContext.ViewData); writer.Write(parsedcontents); }

public string Parse(string contents, ViewDataDictionary viewdata) { return Regex.Replace(contents, "\{(.+)\}", m => GetMatch(m, viewdata)); }

public virtual string GetMatch(Match m, ViewDataDictionary viewdata) { if (m.Success) { string key = m.Result("$1"); if (viewdata.ContainsKey(key)) { return viewdata[key].ToString(); } } return string.Empty; } }
```

How to register Custom View Engine in ASP.NET MVC?

To use your custom View Engine, you need to register it by using global.asax.cs file Application_Start() method, so that the framework will use your custom View Engine instead of the default one.

```
protected void Application_Start() {//Register Custom View Engine ViewEngines.Engines.Add(new CustomViewEngine()); //other code is removed for clarity}
```

Can you remove default View Engine in ASP.NET MVC?

Yes, you can remove default view engines (Razor and WebForm) provided by ASP.NET MVC.

```
protected void Application_Start() { //Remove All View Engine including Webform and Razor ViewEngines.Engines.Clear(); }
```

What are AJAX Helpers?

AJAX Helpers are used to create AJAX enabled elements like as Ajax enabled forms and links which performs request asynchronously. AJAX Helpers are extension methods of AJAXHelper class which exist in System.Web.Mvc.Ajax namespace.

AJAX HTML Element Example

AJAX-enabled linkbased onaction/controller @Ajax.ActionLink("Load Products", "GetProducts", new AjaxOptions {UpdateTargetId = "Products-container", HttpMethod = "GET" }) Output: Load Products

What is unobtrusive AJAX?

ASP.NET MVC supports unobtrusive Ajax which is based on jQuery. The unobtrusive Ajax means that you use helper methods to define your Ajax features, rather than adding blocks of code throughout your views.

What are various configuration options for AJAX Helpers?

The AjaxOptions class defines properties that allow you to specify callbacks for different stages in the AJAX request life cycle. There are following properties provided by AjaxOptions class for AJAX helpers:

Url Specify the URL that will be requested from the server.

Confirm Specify a message that will be displayed in a confirm dialog to the end user. When user clicks on OK button in the confirmation dialog, the Ajax call performs.

OnBegin Specify a JavaScript function name which is called at the beginning of the Ajax request.

OnComplete Specify a JavaScript function name which is called at the end of the Ajax request.

OnSuccess Specify a JavaScript function name which is called when the Ajax request is successful.

OnFailure Specify a JavaScript function name which is called if the Ajax request fails.

LoadingElementId Specify progress message container's Id to display a progress message oranimation to the end user while an Ajax request is being made.

LoadingElementDuration Specify a time duration in milliseconds that controls the duration of the progress message or animation

InsertionMode Specify the way of populating the target container. The possible values are InsertAfter, InsertBefore and Replace (which is the default).

UpdateTargetId Specify the target container's Id that will be populated with the HTML returned bythe action method.

What is Cross Domain AJAX?

By default, web browsers allows AJAX calls only to your web application's site of origin i.e. site hosted server. This restriction help us to prevent various security issues like cross site scripting (XSS) attacks. But, sometimes you need to interact with externally hosted API(s) like Twitter or Google. Hence to interact with these external API(s) or services your web application must support JSONP requests or Cross-Origin Resource Sharing (CORS). By default, ASP.NET MVC does not support JSONP or Cross-Origin Resource Sharing. For this you need to do a little bit of coding and configuration.

What are Layouts in ASP.NET MVC?

Layouts are used to maintain a consistent look and feel across multiple views within ASP.NET MVC application. As compared to Web Forms, layouts serve the same purpose as master pages, but offer a simple syntax and greater flexibility. A basic structure of layout is given below:

@Styles.Render("~/Content/css") @Scripts.Render("~/bundles/modernizr") @RenderBody() @Scripts.Render("~/bundles/jquery") @RenderSection("scripts", required: false)
You can use a layout to define a common template for your site. A layout can be declared at the top of view as: @{ Layout = "~/Views/Shared/SiteLayout.cshtml"; }

What are Sections in ASP.NET MVC?

A section allow you to specify a region of content within a layout. It expects one parameter which is the name of the section. If you don't provide that, an exception will be thrown. A section in a layout page can be defined by using the following code.

@section header{

Header Content

}

You can render above defined section header on the content page as given below:

@RenderSection("header")

By default, sections are mandatory. To make sections optional, just provides the second parameter value as false, which is a Boolean value.

@RenderSection("header",false) Note: A view can define only those sections that are referred to in the layout page otherwise an exception will be thrown.

What are RenderBody and RenderPage in ASP.NET MVC?

RenderBody method exists in the Layout page to render child page/view. It is just like the ContentPlaceHolder on master page. A layout page can have only one RenderBody method.

@RenderBody() @RenderPage("~/Views/Shared/_Header.cshtml") @RenderPage("~/Views/Shared/_Footer.cshtml") @RenderSection("scripts",false)

@section scripts{}

RenderPage method also exists in the Layout page to render other page exists in your application. A layout page can have multiple RenderPage method.

@RenderPage("~/Views/Shared/_Header.cshtml")

What are Styles.Render and Scripts.Render?

Style.Render is used to render a bundle of CSS files defined within BundleConfig.cs files. Styles.Render create style tag(s) for the CSS bundle. Like Style.Render, Scripts.Render is also used to render a bundle of Script files by rendering script tag(s) for the Script bundle.

public class BundleConfig {public static void RegisterBundles(BundleCollection bundles) {bundles.Add(new ScriptBundle("~~/bundles/jqueryval").Include("~~/Scripts/jquery.unobtrusive*", "~~/Scripts/jquery.validate*")); bundles.Add(new StyleBundle("~~/Content/themes/base/css").Include("~~/Content/themes/base/jquery.ui.core.css", "~~/Content/themes/base/jquery.ui.resizable.css", "~~/Content/themes/base/jquery.ui.selectable.css", "~~/Content/themes/base/jquery.ui.button.css", "~~/Content/themes/base/jquery.ui.dialog.css", "~~/Content/themes/base/jquery.ui.theme.css"));} Styles.Render and Scripts.Render generate multiple style and script tags for each item in the CSS bundle and Script bundle when optimizations are disabled. When optimizations are enabled, Styles.Render and Scripts.Render generate a single style and script tag to a version-stamped URL which represents the entire bundle for CSS and Scripts.

How to enable and disable optimizations in ASP.NET MVC?

You can enable and disable optimizations by setting EnableOptimizations property of BundleTable class to true or false with in Global.asax.cs file as shown below. protected void Application_Start() {//other code has been removed for clarity //disable optimization System.Web.Optimization.BundleTable.EnableOptimizations = false;}

What is ViewStart?

_ViewStart.cshml page is used to serve common layout page(s) for a group of views. The code within this file is executed before the code in any view placed in the same directory. This file is also recursively applied to any view within a subdirectory. By default ASP.NET MVC project has a _ViewStart.cshtml file in the Views directory and it specifies a default layout for your ASP.NET MVC application as shown below: @{Layout = "~/Views/Shared/Layout.cshtml";} Since this code runs before any view, hence a view can override the Layout property and choose a different layout.

When to use _ViewStart?

When a set of views shares common settings, the _ViewStart.cshtml file is a great place to put these common view settings. If any view needs to override any of the common settings then that view can set new values to common settings.

What are different ways of rendering layout in ASP.NET MVC?

There are following four different ways of rendering layout in ASP.NET MVC:

Using _ViewStart file in the root directory of the Views folder: The _ViewStart file with in Views folder is used to server the default Layout page for your ASP.NET MVC application. You can also change the default rendering of layouts with in _ViewStart file based on controller as shown below:

@{var controller = HttpContext.Current.Request.RequestContext.RouteData.Values["Controller"].ToSt ring(); string layout = ""; if (controller == "Admin") { layout = "~~/Views/Shared/_AdminLayout.cshtml"; } else { layout = ~~"/Views/Shared/_Layout.cshtml"; } Layout = layout; }

Adding _ViewStart file in each of the directories You can also set the default layout for a particular directory by putting _ViewStart file in each of the directories with the required Layout information as shown below:

Defining Layout with in each view on the top

@{Layout = "~/Views/Shared/_AdminLayout.cshtml";}

Returning Layout from ActionResult

public ActionResult Index() { RegisterModel model = new RegisterModel(); //TO DO: return View("Index", "_AdminLayout", model); }

What is App_Start folder in ASP.NET MVC?

App_Start folder has been introduced in MVC4. It contains various configurations files like as BundleConfig.cs, FilterConfig.cs, RouteConfig.cs, WebApiConfig.cs for your application. All these settings are registered within Application_Start method of Global.asax.cs file.

BundleConfig.cs - This is used to create and register bundles for CSS and JS files. By default, various bundles are added in this files including jQuery, jQueryUI, jQuery validation, Modernizr, and Site CSS.

FilterConfig.cs - This is used to register global MVC filters like error filters, actions filters etc. By default it contains HandleErrorAttribute filter.

RouteConfig.cs - This is used to register various route patterns for your ASP.NET MVC application. By default, one route is registered here named as Default Route.

WebApiConfig.cs - This is used to register various WEB API routes like as ASP.NET MVC, as well as set any additional WEB API configuration settings.

What are different ways of returning/rendering a view in ASP.NET MVC?

There are four different ways for returning/rendering a view in ASP.NET MVC as given below:

Return View() - This tells MVC to generate HTML to be displayed for the specified view and sends it to the browser. This acts like as Server.Transfer() in ASP.NET WebForm.

Return RedirectToAction() - This tells MVC to redirect to specified action instead of rendering HTML. In this case, browser receives the redirect notification and make a new request for the specified action. This acts like as Response.Redirect() in ASP.NET WebForm. Moreover, RedirectToAction construct a redirect url to a specific action/controller in your application and use the route table to generate the correct URL. RedirectToAction cause the browser to receive a 302 redirect within your application and gives you an easier way to work with your route table.

Return Redirect() - This tells MVC to redirect to specified URL instead of rendering HTML. In this case, browser receives the redirect notification and make a new request for the specified URL. This also acts like as Response.Redirect() in ASP.NET WebForm. In this case, you have to specify the full URL to redirect.

Moreover, Redirect also cause the browser to receive a 302 redirect within your application, but

you have to construct the URLs yourself.

Return RedirectToRoute() - This tells MVC to look up the specifies route into the Route table that is defined in global.asax and then redirect to that controller/action defined in that route. This also make a new request like RedirectToAction().

Note:

Return View doesn't make a new requests, it just renders the view without changing URLs in the browser's address bar. Return RedirectToAction makes a new requests and URL in the browser's address bar is updated with the generated URL by MVC. Return Redirect also makes a new requests and URL in the browser's address bar is updated, but you have to specify the full URL to redirect

Between RedirectToAction and Redirect, best practice is to use RedirectToAction for anything dealing with your application actions/controllers. If you use Redirect and provide the URL, you'll need to modify those URLs manually when you change the route table.

RedirectToRoute redirects to a specific route defined in the Route table.

How to persist data in TempData?

The life of TempData is very short and lies only till the target view is fully loaded. But you can persist data in TempData by calling Keep() method after request completion

void Keep() - Calling this method with in the current action ensures that all the items in TempData are not removed at the end of the current request.

public ActionResult Index() { ViewBag.Message = TempData["Message"]; Employee emp = TempData["emp"] as Employee; //need type casting TempData.Keep(); //persist all strings values return View();
}

void Keep(string key) - Calling this method with in the current action ensures that specific item in TempData is not removed at the end of the current request.

public ActionResult Index() { ViewBag.Message = TempData["Message"]; Employee emp = TempData["emp"] as Employee; //need type casting //persist only data for emp key and Message key will be destroy TempData.Keep("emp"); return View(); }

How to control Session behavior in ASP.NET MVC?

By default, ASP.NET MVC support session state. Session is used to store data values across requests. Whether you store some data values with in the session or not ASP.NET MVC must manage the session state for all the controllers in your application that is time consuming. Since, session is stored on server side and consumes server memory, hence it also affect your application performance.

If some of the controllers of your ASP.NET MVC application are not using session state features, you can disable session for those controller and can gain slight performance improvement of your application. You can simplify session state for your application by using available options for session state. In ASP.NET MVC4, SessionState attribute provides you more control over the behavior of session-state by specifying the value of SessionStateBehavior enumeration as shown below:

Default The default ASP.NET behavior is used to determine the session state behavior.

Disabled Session state is disabled entirely.

ReadOnly Read-only session state behavior is enabled.

Required Full read-write session state behavior is enabled.

How TempData is related to Session in ASP.NET MVC?

In ASP.NET MVC, TempData use session state for storing the data values across requests. Hence, when you will disabled the session state for the controller, it will throw the exception as shown below:

What is ActionResult and how is it different from others?

The ActionResult class is the base class for all action results. An action result can be of type ViewResult, JsonResult, RedirectResult and so on. Hence, when your action method returns multiple results based on different conditions, ActionResult is the best choice. Since it can return any type of result.

public ActionResult Index(int id) {if (id == 1) return View(); // returns simple ViewResult else if (id == 2) return Json(new { result = "1" }, JsonRequestBehavior.AllowGet); // returns JsonResult else return RedirectToAction("Login"); // returns to Login Page }

How to make a Non-Action method in ASP.NET MVC?

By default, the ASP.NET MVC framework treats all public methods of a controller class as action methods. If you do not want a public method to be an action method, you must mark that method with the

NonActionAttribute attribute. [NonAction] public void DoSomething() { // Method logic }

Can you change action method name?

You can also change action method name by using ActionName attribute. Now action method will be called by the name defined by the ActionName attribute.

Now, DoSomething action will be identified and called by the name DoAction.

[ActionName("D###oAction")] public ActionResult DoSomething() { //TODO: return View(); }

How to restrict an action method to be invoked only by HTTP GET, POST, PUT or DELETE?

By default, each and every action method can be invoked by any HTTP request (i.e. GET, PUT, POST, and DELETE). But you can restrict an action to be invoked only by a specific HTTP request by applying HttpGet or HttpPost or HttpPut or HttpDelete attribute.

If you want to restrict an action method for HTTP Get request only then decorate it with HttpGet action method selector attribute as given below:

[HttpGet] public ActionResult Index() { //TODO: return View(); }

How to determine an action method is invoked by HTTP GET or POST ?

By using HttpMethod property of HttpRequestBase class, you can find out whether an action is invoked by HTTP GET or POST.

public ActionResult Index(int? id) { if (Request.HttpMethod == "GET") { //TODO: } else if (Request.HttpMethod == "POST") { //TODO: } else { //TODO: } return View(); }

How to determine an AJAX request ?

You can determine an AJAX request by using Request.IsAjaxRequest() method. It will return true, if the request is an AJAX request else returns false.

public ActionResult DoSomething() { if (Request.IsAjaxRequest()) { //TODO: } return View(); }

What is Data Annotations in ASP.NET MVC?

Data validation is a key aspect for developing web application. In Asp.net MVC, we can easily apply validation to web application by using Data Annotation attribute classes to model class. Data Annotation attribute classes are present in

System.ComponentModel.DataAnnotations namespace and are available to Asp.net projects like Asp.net web application & website, Asp.net MVC, Web forms and also to Entity framework ORM models.

Data Annotations help us to define the rules to the model classes or properties for data validation and displaying suitable messages to end users.

Data Annotation Validator Attributes

DataType - Specify the datatype of a property

DisplayName - specify the display name for a property.

DisplayFormat - specify the display format for a property like different format for Date property.

Required - Specify a property as required.

ReqularExpression - validate the value of a property by specified regular expression pattern.

Range - validate the value of a property within a specified range of values.

StringLength - specify min and max length for a string property.

MaxLength - specify max length for a string property.

Bind - specify fields to include or exclude when adding parameter or form values to model properties.

ScaffoldColumn - specify fields for hiding from editor forms.

How to apply Server side validation in ASP.NET MVC ?

Server side validations are very important before playing with sensitive information of a user. Server-side validation must be done whether we validate the received data on the client side. User could disable script in his browser or do something else to bypass client-side validation. In this case server-side validation must require to protect our data from dirty input.

In ASP.NET MVC, there are two ways to validate a model on server side:

Explicit Model Validation – This is the traditional way to validate the model data by using IF..Else..IF statement. In this way, you need to check your model property values one by one for your desired result. If model property values are unexpected, inject error messages within ModelState.

class HomeController : Controller { [HttpPost] public ActionResult ExplicitServer(UserViewModel model) { //Write custom logic to validate UserViewModel if (string.IsNullOrEmpty(model.UserName)) { ModelState.AddModelError("UserName", "Please enter your name"); } if (!string.IsNullOrEmpty(model.UserName)) { Regex emailRegex = new Regex(".+@.+\..+"); if (!emailRegex.IsMatch(model.UserName)) ModelState.AddModelError("UserName", "Please enter correct email address"); } if (ModelState.IsValid) //Check model state { //TO DO: } } }

Model Validation with Data Annotations - Data Annotations was introduced with .NET 3.5 SP1. It has a set of attributes and classes defined in the System.ComponentModel.DataAnnotations assembly. Data Annotations allow us to decorate model classes with metadata. This metadata describes a set of rules that are used to validate a property.

describes a set of rules that are used to validate a property.

public class UserViewModel {[Required(ErrorMessage = "Please Enter Email Address")] [RegularExpression(".+@.+\..+", ErrorMessage = "Please Enter CorrectEmail Address")] public string UserName { get; set; } [Required(ErrorMessage = "Please Enter Password")] [StringLength(50, ErrorMessage = "The {0} must be at least {2}characters long.", MinimumLength = 6)] public string Password { get; set; } }

How to determine there is no error in Modal State ?

When server side model validation fails, errors are included in the ModelState. Hence, by using ModelState.IsValid property you can verify model state. It returns true if there is no error in ModelState else returns false.

[HttpPost] public ActionResult DoSomething(UserViewModel model) {if (ModelState.IsValid) {//TODO:} return View();}

How to enable and disable client-side validation in ASP.NET MVC ?

We can enable and disable the client-side validation by setting the values of ClientValidationEnabled & UnobtrusiveJavaScriptEnabled keys true or false. This setting will be applied to application level.

For client-side validation, the values of above both the keys must be true. When we create new project using Visual Studio in MVC3 or MVC4, by default the values of both the keys are set to true.

We can also enable the client-side validation programmatically. For this we need to do code with in the Application_Start() event of the Global.asax, as shown below.

protected void Application_Start()

{//Enable or Disable Client Side Validation at Application Level HtmlHelper.ClientValidationEnabled = true; HtmlHelper.UnobtrusiveJavaScriptEnabled = true;}

We can also enable or disable client-side validation for a specific view. For this we required to enable or disable client side validation inside a Razor code block as shown below. This option will overrides the application level settings for that specific view. @using MvcApp.Models @{ViewBag.Title = "About"; HtmlHelper.ClientValidationEnabled = false;}

What is a CDN and advantage of CDN?

CDN stands for content delivery network or content distribution network (CDN) which is a large distributed system of servers deployed in multiple data centers across the Internet. The goal of a CDN is to serve the content (like jQuery library and other open source libraries) to end-users with high availability and high performance.

There are three popular CDN – Google, Microsoft and jQuery. // Google CDN

<scripttype="text/javascript"src="http://ajax.googleapis.com/ajax/libs/jquery/1.9.1/jquery.min.js"> // Microsoft CDN

<scripttype="text/javascript"src="http://ajax.microsoft.com/ajax/jquery/jquery-1.9.1.min.js"> // JQuery CDN

<scripttype="text/javascript"src="http://code.jquery.com/jquery-1.9.1.min.js">

Advantages

1. It reduces the load from your application server.

It saves bandwidth since jQuery and other open libraries/framework will load faster from these CDN. The most important benefit is it will be cached means if a user has visited any site which

CDN. The most important benefit is it will be cached means if a user has visited any site which is using jQuery framework from any of these CDN and your web application is also using the same CDN for serving the jQuery then for your application, it will not request the jQuery from CDN.

What is Jquery.validate.unobtrusive.js?

Or

What is jQuery Validation Unobtrusive plugin?

Microsoft introduced jquery.validate.unobtrusive.js plugin with ASP.NET MVC3 to apply data model validations to the client side using a combination of jQuery Validation and HTML 5 data attributes.

Can we use Bundling and minification in ASP.NET MVC3 or ASP.NET4.0?

System.Web.Optimization class offers the bundling and minification techniques that is exist within the Microsoft.Web.Optimization dll. Using this dll you can also use this technique with ASP.NET MVC3 and .NET Framework 4.0.

How Bundling use browser Cache capability?

Browsers cache resources based on URLs. When a web page requests a resource, the browser first checks its cache to see if there is a resource with the matched URL. If yes, then it simply uses the cached copy instead of fetching a new one from server. Hence whenever you change the content of CSS and JS files will not reflect on the browser. For this you need to force the browser for refreshing/reloading.

But bundles automatically takes care of this problem by adding a hash code to each bundle as a query parameter to the URL as shown below. Whenever you change the content of CSS and JS files then a new has code will be generated and rendered to the page automatically. In this way, the browser will see a different Url and will fetch the new copy of CSS and JS.

How do you return a partial view from controller?

return PartialView(options); where options could be a Model or a View name

What is difference ways of rendering a Partial View in ASP.NET MVC?

There are four methods for rendering a partial view in ASP.NET MVC These are RenderPartial, RenderAction, Partial and Action helper methods.

Html.RenderPartial

This method result will be directly written to the HTTP response stream means it used the same TextWriter object as used in the current webpage/template.This method returns void.

Simple to use and no need to create any action.

RenderPartial method is useful used when the displaying data in the partial view is already in the corresponding view model. For example: In a blog to show comments of an article, we would like to use RenderPartial method since an article information with comments are already populated in the view model.

This method is faster than Partial method since its result is directly written to the response stream which makes it fast. @{Html.RenderPartial("_Comments");}

Html.RenderAction

This method result will be directly written to the HTTP response stream means it used the same TextWriter object as used in the current webpage/template.

For this method, we need to create a child action for the rendering the partial view.

RenderAction method is useful when the displaying data in the partial view is independent from corresponding view model. For example: In a blog to show category list on each and every page, we would like to use RenderAction method since the list of category is populated by the different model. @{Html.RenderAction("Category","Home");} This method is the best choice when you want to cache a partial view. This method is faster than Action method since its result is directly written to the HTTP response stream which makes it fast.

Html.Partial

Renders the partial view as an HTML-encoded string.

This method result can be stored in a variable, since it returns string type value.

Simple to use and no need to create any action.

Partial method is useful used when the displaying data in the partial view is already in the corresponding view model. For example: In a blog to show comments of an article, we would like to use RenderPartial method since an article information with comments are already populated in the view model.

@Html.Partial("_Comments")

Html.Action

Renders the partial view as an HtmlString .

For this method, we need to create a child action for the rendering the partial view.

This method result can be stored in a variable, since it returns string type value.

Action method is useful when the displaying data in the partial view is independent from corresponding view model. For example: In a blog to show category list on each and every page, we would like to use Action method since the list of category is populated by the different model.

@{Html.Action("Category","Home");}

This method is also the best choice when you want to cache a partial view.

How to register Area in ASP.NET MVC?

Before working with area, make sure you have registered your area with in the Application_Start method in Global.asax as shown below.

protected void Application_Start() {//Register all application Areas AreaRegistration.RegisterAllAreas(); }

Always remember the order of registering the Areas must be on top, so that all of the settings, filters and routes registered for the applications will also apply on the Areas.

What is child action and how to invoke it?

Child actions are useful for creating reusable widgets which could be embedded into your views. In ASP.NET MVC partial views are used to create reusable widgets and a partial can be render by an action method.

This action method can has child attribute and has its independent MVC lifecycle from parent view. Also, an action which has child attribute cannot be called independently. It always will be called within a parent view otherwise it would give error.

[ChildActionOnly] public ActionResult MenuBar() { //TODO: return PartialView(); }

A child action is invoked by using @Html.RenderAction or @Html.Action helper methods from inside of a view.

What is Scaffolding?

Scaffolding is a technique used by many MVC frameworks like ASP.NET MVC, Ruby on Rails, Cake PHP and Node.JS etc., to generate code for basic CRUD (create, read, update, and delete) operations against your database effectively. Further you can edit or customize this auto generated code according to your need.

Scaffolding consists of page templates, entity page templates, field page templates, and filter templates. These templates are called Scaffold templates and allow you to quickly build a functional data-driven Web site.

How Scaffold template works in ASP.NET MVC?

Scaffold templates are used to generate code for basic CRUD operations within your ASP.NET MVC applications against your database with the help Entity Framework. These templates use the Visual Studio T4 templating system to generate views for basic CRUD operations with the help of Entity Framework.

Steps to create ASP.NET MVC CRUD operations using scaffolding in ASP.NET MVC:

Step1: Adding controller to your project

Step2: Choosing a scaffold template for creating CRUD operations

Step3: Provide a name to your controller

The following actions are created for insert, update and delete operations based on scaffold template within User controller.

What are ASP.NET MVC Filters and Attributes ?

ASP.NET MVC provides a simple way to inject your piece of code or logic either before or after an action is executed. This is achieved by decorating the controllers or actions with ASP.NET MVC attributes or custom attributes. An attribute or custom attribute implements the ASP.NET MVC filters (filter interface) and can contain your piece of code or logic. You can make your own custom filters or attributes either by implementing ASP.NET MVC filter interface or by inheriting and overriding methods of ASP.NET MVC filter attribute class if available.

Typically, Filters are used to perform the following common functionalities in your ASP.NET MVC application.

Custom Authentication Custom Authorization (User based or Role based)

Error handling or logging User Activity Logging

Data Caching Data Compression

What are difference types of Filters in ASP.NET MVC ?

The ASP.NET MVC framework provides five types of filters.

Authentication Filters - This filter is introduced with ASP.NET MVC5. The IAuthenticationFilter interface is used to create CustomAuthentication filter. The definition of this interface is given below-

public interface IAuthenticationFilter { void OnAuthentication(AuthenticationContext filterContext); void OnAuthenticationChallenge(AuthenticationChallengeContext filterContext); }

filterContext); void OnAuthenticationChallenge(AuthenticationChallengeContext filterContext); }

You can create your CustomAuthentication filter attribute by implementing IAuthenticationFilter as shown below-

public class CustomAuthenticationFilterAttribute : FilterAttribute, IAuthenticationFilter { public void OnAuthentication(AuthenticationContext filterContext) { filterContext.HttpContext.Response.Write("Authentication Filter "); } //Runs after the OnAuthentication method

public void OnAuthenticationChallenge(AuthenticationChallengeContext filterContext) { //TODO: Additional tasks on the request } }

Authorization Filters - The ASP.NET MVC Authorize filter attribute implements the IAuthorizationFilter interface. The definition of this interface is given below-

public interface IAuthorizationFilter { void OnAuthorization(AuthorizationContext filterContext); }

The AuthorizeAttribute class provides the following methods to override in the CustomAuthorize attribute class.

public class AuthorizeAttribute : FilterAttribute, IAuthorizationFilter { protected virtual bool AuthorizeCore(HttpContextBase httpContext); protected virtual void HandleUnauthorizedRequest(AuthorizationContext filterContext); public virtual void OnAuthorization(AuthorizationContext filterContext); protected virtual HttpValidationStatus OnCacheAuthorization(HttpContextBase httpContext); }

In this way you can make your CustomAuthorize filter attribute either by implementing IAuthorizationFilter interface or by inheriting and overriding above methods of AuthorizeAttribute class.

Action Filters - Action filters are executed before or after an action is executed. The IActionFilter interface is used to create an Action Filter which provides two methods OnActionExecuting and OnActionExecuted which will be executed before or after an action is executed respectively.

public interface IActionFilter { void OnActionExecuting(ActionExecutingContext filterContext); void OnActionExecuted(ActionExecutedContext filterContext); }

Result Filters - Result filters are executed before or after generating the result for an action. The Action Result type can be ViewResult, PartialViewResult, RedirectToRouteResult, RedirectResult, ContentResult, JsonResult, FileResult and EmptyResult which derives from the ActionResult class. Result filters are called after the Action filters. The IResultFilter interface is used to create a Result Filter which provides two methods OnResultExecuting and OnResultExecuted which will be executed before or after generating the result for an action respectively.

public interface IResultFilter { void OnResultExecuted(ResultExecutedContext filterContext); void OnResultExecuting(ResultExecutingContext filterContext); }

Exception Filters - Exception filters are executed when exception occurs during the actions execution or filters execution. The IExceptionFilter interface is used to create an Exception Filter which provides OnException method which will be executed when exception occurs during the actions execution or filters execution.

public interface IExceptionFilter { void OnException(ExceptionContext filterContext); }

The HandleErrorAttribute class is one example of an exception filter which implements IExceptionFilter. When HandleError filter receives the exception it returns an Error view located in the Views/Shared folder of your ASP.NET MVC application.

When Exception Filters are executed in ASP.NET MVC?

Exception filters are executed if there is an unhandled exception thrown during the execution of

the ASP.NET MVC pipeline.

What is the order of execution of filters in ASP.NET MVC?

All ASP.NET MVC filter are executed in an order. The correct order of execution is given below:

Authentication filters

Authorization filters

Action filters

Result filters

How to configure Filters in ASP.NET MVC ?

You can configure your own custom filter into your application at following three levels:

Global level - By registering your filter into Application_Start event of Global.asax.cs file with the help of FilterConfig class.

protected void Application_Start(){ FilterConfig.RegisterGlobalFilters(GlobalFilters.Filters);} Controller level - By putting your filter on the top of the controller name as shown below- [Authorize(Roles = "Admin")] public class AdminController : Controller { //TODO: }

Action level - By putting your filter on the top of the action name as

shown below- public class UserController : Controller {[Authorize(Users = "User1,User2")] public ActionResult LinkLogin(string provider) { // TODO: return View(); } }

How Authentication and authorization work in ASP.NET MVC ?

Like ASP.NET, MVC also supports Windows and Forms authentication. You can configure both the authentications by using Web.config or doing some custom code.

How Forms Authentication and authorization work in ASP.NET MVC?

Like ASP.NET, MVC Forms authentication occurs after IIS authentication is completed. It can be configure by using forms element within Web.config file of your ASP.NET MVC application. The default attribute values for forms authentication are shown below:

<system.web> </system.web>

The FormsAuthentication class creates the authentication cookie automatically when SetAuthCookie() or RedirectFromLoginPage() methods are called. The value of authentication cookie contains a string representation of the encrypted and signed FormsAuthenticationTicket object.

You can create the FormsAuthenticationTicket object by specifying the cookie name, version of the cookie, directory path, issue date of the cookie, expiration date of the cookie, whether the cookie should be persisted, and optionally user-defined data as shown below:

FormsAuthenticationTicket ticket = new FormsAuthenticationTicket(1, "userName", DateTime.Now, DateTime.Now.AddMinutes(30), // value of time out property false, // Value of IsPersistent property String.Empty, FormsAuthentication.FormsCookiePath); Now, you can encrypt this ticket by using the Encrypt method FormsAuthentication class as given below: string encryptedTicket = FormsAuthentication.Encrypt(ticket);

How to implement custom Forms Authentication and authorization in MVC ?

When standard types of authentication do not meet your requirements, you need to modify an

When standard types of authentication do not meet your requirements, you need to modify an authentication mechanism to create a custom solution. A user context has principal which represents the identity and roles for that user. A user is authenticated by its identity and assigned roles to a user determine about authorization or permission to access resources.

ASP.NET provides IPrincipal and IIdentity interfaces to represents the identity and role for a user. You can create a custom solution by evaluating the IPrincipal and IIdentity interfaces which are bound to the HttpContext as well as the current thread.

public class CustomPrincipal : IPrincipal { public IIdentity Identity { get; private set; } public bool IsInRole(string role) { if (roles.Any(r => role.Contains(r))) { return true; } else { return false; } } public CustomPrincipal(string Username) { this.Identity = new GenericIdentity(Username); } public int UserId { get; set; } public string FirstName { get; set; } public string LastName { get; set; } public string[] roles { get; set; } }

Now you can put this CustomPrincipal objects into the thread's CurrentPrincipal property and into the HttpContext's User property to accomplish your custom authentication and authorization process.

A user will be authenticated if IsAuthenticated property returns true. For authenticating a user you can use one of the following two ways:

Thread.CurrentPrincipal.Identity.IsAuthenticated

HttpContext.Current.User.Identity.IsAuthenticated

ASP.NET MVC provides Authorization filter to authorize a user. This filter can be applied to an action, a controller, or even globally. This filter is based on AuthorizeAttribute class. You can customize this filter by overriding OnAuthorization() method as shown below:

public class CustomAuthorizeAttribute : AuthorizeAttribute { protected virtual CustomPrincipal CurrentUser { get { return HttpContext.Current.User as CustomPrincipal; } } public override void OnAuthorization(AuthorizationContext filterContext) { if (filterContext.HttpContext.Request.IsAuthenticated) { if (!String.IsNullOrEmpty(Roles)) { if (!CurrentUser.IsInRole(Roles)) { filterContext.Result = new RedirectToRouteResult(new RouteValueDictionary(new { controller = "Error", action = "AccessDenied" })); // base.OnAuthorization(filterContext); //returns to login url } } if (!String.IsNullOrEmpty(Users)) { if (!Users.Contains(CurrentUser.UserId.ToString())) {filterContext.Result = new RedirectToRouteResult(new RouteValueDictionary(new { controller = "Error", action = "AccessDenied" })); // base.OnAuthorization(filterContext); //returns to login url } } } } }

Now you can apply this custom authorization filter at controller or action level for authorization as shown below:

[CustomAuthorize(Roles= "Admin")] public class AdminController : BaseController { public ActionResult Index() { return View(); } }

How to allow HTML tags in ASP.NET MVC?

By default ASP.NET MVC doesn't allow a user to submit html for avoiding Cross Site Scripting attack to your application. You can achieve it by using ValidateInput attribute and AllowHtml attribute.

ValidateInput attribute can enable or disable input validation at the controller level or at any action method.

[ValidateInput(false)] public class HomeController : Controller { public ActionResult AddArticle() { return View(); } }

ValidateInput attribute allow the Html input for all the properties and that is unsafe. Since you have enable Html input for only one-two properties then how to do this. To allow Html input for a single property, you should use AllowHtml attribute.

```
public class BlogModel { [Required] [Display(Name = "Title")] public string Title { get; set; } [AllowHtml] [Required] [Display(Name = "Description")] public string Description { get; set; } }
```

What is caching and when to use it?

Caching is a most important aspect of high-performance web application. Caching provides a way of storing frequently accessed data and reusing that data. Practically, this is an effective way for improving web application's performance.

When to use caching

Use caching for contents that are accessed frequently.

Avoid caching for contents that are unique per user.

Avoid caching for contents that are accessed infrequently/rarely.

Use the VaryByCustom function to cache multiple versions of a page based on customization aspects of the request such as cookies, role, theme, browser, and so on.

For efficient caching use 64-bit version of Windows Server and SQL Server.

For database caching make sure your database server has sufficient RAM otherwise, it may degrade the performance.

For caching of dynamic contents that change frequently, define a short cache–expiration time rather than disabling caching.

What are advantage of caching?

There are following advantages of caching:

Reduce hosting server round-trips

When content is cached at the client or in proxies, it cause minimum request to server.

Reduce database server round-trips

When content is cached at the web server, it can eliminate the database request.

Reduce network traffic

When content is cached at the client side, it also reduce the network traffic.

Avoid time-consumption for regenerating reusable content

When reusable content is cached, it avoid the time consumption for regenerating reusable content.

Improve performance

Since cached content reduce round-trips, network traffic and avoid time consumption for regenerating reusable content which cause a boost in the performance.

What is output caching?

The OutputCache filter allow you to cache the data that is output of an action method. By default, this attribute filter cache the data till 60 seconds. After 60 sec, ASP.NET MVC will execute the action method again and cache the output again.

```
class HomeController : Controller { [OutputCache(Duration = 20, VaryByParam = "none")] public ActionResult Index() { ViewBag.Message = DateTime.Now.ToString(); return View(); } }
```

The output of the Index() action method will be cached for 20 seconds. If you will not defined the duration, it will cached it for by default cache duration 60 sec.

Output Caching Location By default, content is cached in three locations: the web server, any proxy servers, and the user's browser. You can control the content's cached location by changing the location parameter of the OutputCache attribute to any of the following values: Any, Client,Downstream, Server, None, or ServerAndClient.

By default, the location parameter has the value Any which is appropriate for most the scenarios. But sometimes there are scenarios when you required more control over the cached data.

What is donut caching and donut hole caching in ASP.NET MVC?

Donut caching cache an entire web page except for one or more parts of the web page. Before Donut caching, we have Output Caching which cache the entire web page.

When to use Donut caching

Suppose, you have a web application in which some pages like HomePage,Tools etc. are same for all the users excepts the user's logged in details like username.

If you want to cache all these pages for all the users by using OutputCache with VaryByParam UserID, then the entire page would be cached every time for each user with a different user name (or whatever your dynamic part of the page is). This is not a good practice since there will be 1000 cached pages if there are 1000 logged in user at a time.

To resolve this issue, Donut Caching was introduced which cached only one copy of the entire page for all the user except for a small part which remain dynamic. This small part act like as a hole in the cached content and much like a donut.

Donut caching is very useful in the scenarios where most of the elements in your page are rarely changed except the few sections that dynamically change, or changed based on a request parameter.

Donut Hole caching Donut Hole Caching is the inverse of Donut caching means while caching the entire page it cached only a small part of the page (the donut hole).

When to use Donut Hole caching Suppose, you have a web application in which ProductCategory is shown on each and every pages so it makes sense to render all of the categories just once and cache the resulting HTML by using Donut Hole Caching.

Donut Hole caching is very useful in the scenarios where most of the elements in your page are dynamic except the few sections that rarely change, or changed based on a request parameter. ASP.NET MVC has great support for Donut Hole caching through the use of Child Actions.

class HomeController : Controller { [ChildActionOnly] [OutputCache(Duration = 60)] public ActionResult CategoriesList() { // Get categories list from the database and // pass it to the child view ViewBag.Categories = GetCategories(); return View(); } }

What is loose coupling and how is it possible?

One of the most important features of the MVC design pattern is that it enables separation of concerns. Hence you can make your application’s components independent as much as possible. This is known as loose coupling, and it makes testing and maintenance of our application easier. Using Dependency Injection you can make you application’s components more loosely coupled.

What are Dependency Inversion Principle(DIP) and IoC ?

The Dependency Inversion Principle states that:

High level modules should not depend upon low level modules. Both should depend upon abstractions.

Abstractions should not depend upon details. Details should depend upon abstractions.

The Dependency Inversion principle (DIP) helps us to develop loosely couple code by ensuring that high-level modules depend on abstractions rather than concrete implementations of lower-level modules. The Inversion of Control pattern is an implementation of this principle.

The term Inversion of Control (IoC) refers to a programming style where a framework or runtime, controls the program flow. Inversion of control means we are changing the control from normal way. It works on Dependency Inversion Principle. The most software developed on the .NET Framework uses IoC.

More over IoC is a generic term and it is not limited to DI. Actually, DI and Service Locator patterns are specialized versions of the IoC pattern or you can say DI and Service Locator are the ways of implementing IoC.

For example, suppose your Client class needs to use a Service class component, then the best you can do is to make your Client class aware of an IService interface rather than a Service class. In this way, you can change the implementation of the Service class at any time (and for how many times you want) without breaking the host code.

IoC and DIP

DIP says High level module should not depend on low level module and both should depend on abstraction. IoC is a way that provide abstraction. A way to change the control. IoC gives some ways to implement DIP. If you want to make independent higher level module from the lower level module then you have to invert the control so that low level module do not control interface and creation of object. Finally IoC gives some way to invert the control.

What is Dependency Injection(DI)?

DI is a software design pattern that allow us to develop loosely coupled code. DI is a great way to reduce tight coupling between software components. DI also enables us to better manage future changes and other complexity in our software. The purpose of DI is to make code maintainable.

The Dependency Injection pattern uses a builder object to initialize objects and provide the required dependencies to the object means it allows you to "inject" a dependency from outside the class.

For example, suppose your Client class needs to use a Service class component, then the best you can do is to make your Client class aware of an IService interface rather than a Service class. In this way, you can change the implementation of the Service class at any time (and for how many times you want) without breaking the host code.

DI Implementation:

What is Service Locator?

Service Locator is a software design pattern that also allow us to develop loosely coupled code. It implements the DIP principle and easier to use with an existing codebase as it makes the overall design looser without forcing changes to the public interface. The Service Locator pattern introduces a locator object that objects is used to resolve dependencies means it allows you to "resolve" a dependency within a class. Above example can be re-written as follows by using SL.

public interface IService { void Serve(); } public class Service : IService { public void Serve() { Console.WriteLine("Service Called"); //To Do: Some Stuff } }

public static class LocateService { public static IService _Service { get; set; } public static

```
IService GetService() { if (_Service == null) _Service = new Service(); return _Service; } } public class Client { private IService _service; public Client() { this._service = LocateService.GetService(); } public void Start() { Console.WriteLine("Service Started"); this._service.Serve(); //To Do: Some Stuff } } class Program { static void Main(string[] args) { var client = new Client(); client.Start(); Console.ReadKey(); } }
```

What are difference ways to implement Dependency Injection(DI)?

There are three different ways to implement DI as given below:

Constructor Injection - This is the most common DI. Dependency Injection is done by supplying the DEPENDENCY through the class's constructor when instantiating that class. Injected component can be used anywhere within the class. Should be used when the injected dependency is required for the class to function. It addresses the most common scenario where a class requires one or more dependencies.

```
public interface IService { void Serve(); } public class Service : IService { public void Serve() { Console.WriteLine("Service Called"); //To Do: Some Stuff } } public class Client { private IService _service; public Client(IService service) { this._service = service; } public void Start() { Console.WriteLine("Service Started"); this._service.Serve(); //To Do: Some Stuff } } //Builder class Program { static void Main(string[] args) { Client client = new Client(new Service()); client.Start(); Console.ReadKey(); } }
```

Property Injection – This is also called Setter injection. This is used when a class has optional dependencies, or where the implementations may need to be swapped. This is used by different logger implementations like Log4Net. It may require checking for a provided implementation throughout the class (need to check for null before using it). It does not require adding or modifying constructors.

```
public interface IService { void Serve(); } public class Service : IService { public void Serve() { Console.WriteLine("Service Called"); //To Do: Some Stuff } } public class Client { private IService _service; public IService Service { set { this._service = value; } } public void Start() {Console.WriteLine("Service Started"); this._service.Serve(); //To Do: Some Stuff }} //Builder class Program { static void Main(string[] args) { Client client = new Client(); client.Service = new Service(); client.Start(); Console.ReadKey(); } }
```

Method Injection – This Inject the dependency into a single method, for use by that method only. It could be useful where the whole class does not need the dependency, just the one method.

```
public interface IService { void Serve(); } public class Service : IService { public void Serve() { Console.WriteLine("Service Called"); //To Do: Some Stuff } }

public class Client { private IService _service; public void Start(IService service) { this._service = service; Console.WriteLine("Service Started"); this._service.Serve(); //To Do: Some Stuff } } //Builder class Program { static void Main(string[] args) { Client client = new Client(); client.Start(new Service()); Console.ReadKey(); } }
```

What are advantages of Dependency Injection(DI)?

There are following advantages of DI: Reduces class coupling Increases code reusing Improves code maintainability Improves application testing

What is IoC or DI container?

The terms Dependency Injection (DI) & Inversion of Control (IoC) are generally used interchangeably to describe the same design pattern. Hence some people says IoC Container and some people says DI container but both terms indicate to the same thing. So don't be confused from the terminology.

A DI Container is a framework to create dependencies and inject them automatically when required. It automatically creates objects based on request and inject them when required. DI

required. It automatically creates objects based on request and inject them when required. DI Container helps us to manage dependencies with in the application in a simple and easy way.

We can also manage an application dependencies without a DI Container, but it will be like as POOR MAN'S DI and we have to do more work, to make it configured and manageable.

What are popular DI containers?

Today, there are a lot of excellent DI Containers that are available for .NET. The list of most useful DI container for .NET framework is given below:

Castle Windsor

Based on the Castle MicroKernel.

Well documented and used by many.

Understands Decorator

Typed factories

Commercial support available

Spring.NET

INTERCEPTION

Comprehensive documentation

Commercial support available

Autofac

Easy to learn API

second-generation DI Container

Commercial support available

Unity

INTERCEPTION

Good documentation

Consistent API

Ninject

Easy to learn API

Second-generation DI Container

What is Test Driven Development(TDD)?

TDD is a methodology which says, write your tests first before you write your code. In TDD, tests drive your application design and development cycles. You do not do the check-in of your code into source control until all of your unit tests pass.

What are commonly used tool for Unit Testing in ASP.NET MVC?

ASP.NET MVC has been designed for testability without dependencies on the IIS server, on a database, or on external classes. There are following popular tools for ASP.NET MVC testing:

NUnit - This is the most popular unit testing frameworks for Microsoft .NET. Its syntax is relatively simple and easy to use. It comes with a test runner GUI and a command-line utility. NUnit is also available as a NuGet package for download.

xUnit.NET - This provides a way to run automated unit tests. It is simple, easily extended, and has a very clean syntax.

Ninject 2 - This provides a way to wire up classes in your application.

Moq - This provides a framework for mocking interfaces and classes during testing.